



Octopus

A cycle-accurate memory-system simulator
for predictable multicores

Reference manual

Fanos Research Lab · McMaster University

Revision 2c7950e5 · October 1, 2026

<https://github.com/FanosResearch/OctopusSimulator>

About this manual

To be written: what Octopus is, who this manual is for, how to read it, where the sources live.

Contents

About this manual	ii
1 Introduction	1
2 Getting started	2
3 System organization	3
3.1 Design principles	3
3.2 The clock model	4
3.3 Component map	4
3.4 The system as shipped	4
4 The cache controller	8
4.1 One class, every level	8
4.2 From message to actions	9
4.3 The data handler	10
4.4 Admission and back-pressure	10
4.5 The LLC's extra duties	11
4.6 Message encoding	11
4.7 Delivery order at the bus interface	12
5 The last-level cache datapath	13
5.1 The datapath	13
5.2 A read hit	13
5.3 A write hit	16
5.4 A read miss into a free way	16
5.5 A miss that evicts	18
6 The coherence engine: CSV finite-state machines	20
7 A request, end to end	21
7.1 The journey	21
7.2 The stamps	21
7.3 The columns	21
7.4 Checking a column	25

8 Interconnects and arbitration	26
9 Main memory	27
10 Configuration	28
11 Monitoring: Logger, Debugger, trace and visualizer	29
12 Periodic tasks and the interference demo	30
13 Extending Octopus	31
14 Integration modes: gem5 and the full-system stack	32
15 Hands-on tutorial	33
16 Evaluation	34
A Configuration key reference	35
B The shipped coherence FSMs	47
B.1 MSI_splitBus_snooping	47
B.2 MSI_LLC	49
B.3 MESI_splitBus_snooping	51
B.4 MESI_LLC	53
B.5 MOESI_splitBus_snooping	55
B.6 MOESI_LLC	57
B.7 MSI_directory	59
B.8 MSI_LLC_directory	61
C Command line, scripts and environment	63
D File formats	64

List of Figures

3.1	The clock model. One <code>ClockManager</code> advances every <code>ClockedObj</code> by one tick; components exchange messages only through interfaces, which the next component drains on its own tick.	4
3.2	What a configured system can contain: clusters of cores with private caches on direct interconnects, further cache levels inside or across clusters, a shared last-level cache and main memory — all of it chosen by configuration, not hard-coded.	5
3.3	The class diagram. Cache-related classes in red, interconnect-related in blue, utilities in white.	6
3.4	The system <code>MultiCoreSystem.csv</code> builds. Four trace-driven cores with private MESI L1s on a <code>TripleBus</code> , one shared inclusive LLC, a point-to-point memory bus and a fixed-latency main memory. Every number is a configuration key (Chapter A).	6
4.1	Inside a cache controller. Messages from both interfaces are serialised into one processing queue; the protocol handler turns each one into actions by looking the (state, event) pair up in the FSM table that the FSM reader parsed at start-up; the data handler executes those actions against the array, the MSHRs and the write-back buffer, arbitrating the data port.	8
4.2	One message through the controller. The handler decodes the message into an event, looks up the transition for the line's current state, and hands the controller a list of protocol-agnostic actions; the controller does not know which protocol it is running.	9
5.1	The shipped LLC as a datapath: <code>CacheController_End2End</code> with <code>CacheDataHandler_COTS</code> , 32 KB, 2-way, 64 B lines, the MESI_LLC FSM and one timed data port. Bus interfaces on top, control on the left, the single array in the middle read by an untimed bits path and a timed data port on the right, MSHR and write-back buffer beside it, the memory bus below.	14
5.2	A read or write hit — a <code>GetS</code> or <code>GetM</code> that finds the line in a stable state — from the request's arrival on the bus to its data leaving on the response lane, in ten numbered steps.	15
5.3	The write hit on its own. The path is the read hit's; what is new is the branch marked 5b — the FSM's <code>SetOwner</code> written back into the line's bits over the dashed control wire, untimed — and the plain-data response.	16
5.4	A read miss in the simplest case: the set still has a free way, so nothing is evicted. The request's own path in teal; the array write of the fill, off the critical path, dashed.	17

5.5	A miss into a <i>full</i> set: the requested line in teal, the evicted victim in amber. The victim is chosen when the fill is written, and its exit is a coherence transaction of its own.	18
7.1	The system as shipped with one LLC <i>miss</i> from core 0 routed through it. Each hop carries the colour of the LatencyReport column that measures it (amber: waiting; purple: a bus transfer; teal: being served) and the number of the Logger stamp that bounds it, ①–⑩ being the report's nine latency columns in order; the side panel lists the stamp pair behind every column.	22
7.2	The same system with an LLC <i>hit</i> from core 0. The route stops at the LLC's array port, where the data is emitted at the grant; the memory bus and DRAM are not on the path, so L2-DRAM Bus and DRAM are 0.	22
7.3	The same two routes on a time axis: an LLC hit and an LLC miss, with the nine LatencyReport columns bracketed between the Logger stamps that bound them. Ticks are in order, not to scale. Below the miss axis, the array write of the fill runs alongside the response and lands in no column.	23

Chapter 1

Introduction

What you will find here. To be written.

Chapter 2

Getting started

What you will find here. To be written.

System organization

What you will find here. The four principles the simulator is built on and the two base classes that carry them; the clock model, which is the reason every latency it reports is comparable; the map of components and the classes behind them; and the exact system that `MultiCoreSystem.csv` builds when you run the shipped configuration — the system every later chapter’s figures are drawn on.

3.1 Design principles

Octopus models a multicore memory hierarchy as a *network of clocked, configurable components* joined by bounded, back-pressured *interfaces*. A memory request is a first-class `Message` object that travels this network; a single global clock steps every component; and every hop is timestamped for the `Logger`. Four principles shape the code, and they are realised through two base classes that every component inherits (Table 3.1).

- **Modularity.** Each component is self-contained behind a narrow interface, so changing one does not ripple into others. A cache controller does not know whether it is an L1 or the LLC, nor which protocol it runs; a bus does not know what it carries.
- **Extensibility.** Inheritance and polymorphism: a new arbiter, replacement policy, topology or memory model is a subclass with a handful of virtual methods; a new coherence protocol is a CSV file. The effect shows in the code size of protocols derived from a common family (Chapter 6).
- **Configurability.** Every component derives from `Configurable`: it reads its defaults from its own CSV, a parent may override a child’s, and the command line overrides everything, so the whole design space is reachable without recompiling (Chapter 10).
- **Integrability.** The two endpoints — the request source and the memory sink — are swappable for external simulators: a trace-driven CPU or `gem5` on one side, a fixed-latency model or `MCsim`’s DDR controllers on the other, with the hierarchy between them unchanged (Chapter 14).

Table 3.1. The two base classes every component inherits.

base class	gives every component
<code>ClockedObj</code>	<code>m_clk_period</code> and <code>cycleProcess()</code> : the component can be advanced one tick by the <code>ClockManager</code>
<code>Configurable</code>	<code>name</code> , <code>parent_name</code> , <code>parameters</code> , <code>parseFile()</code> : hierarchical parameters read from CSV and overridable from the CLI

3.2 The clock model

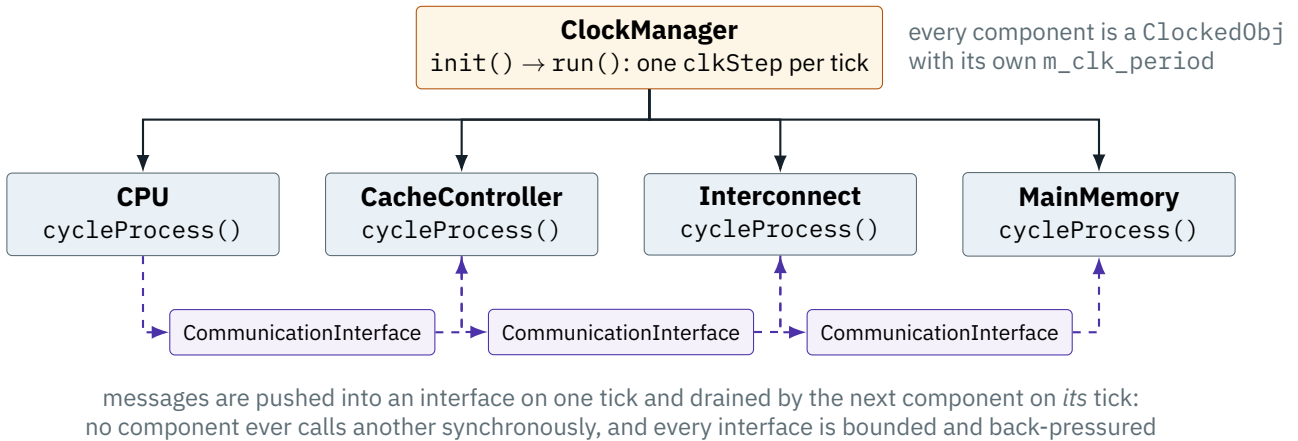


Figure 3.1. The clock model. One ClockManager advances every ClockedObj by one tick; components exchange messages only through interfaces, which the next component drains on its own tick.

The ClockManager owns the simulation loop. On each clkStep it advances every ClockedObj whose period has elapsed by calling its cycleProcess() (Figure 3.1). Components never call each other synchronously: they communicate only by placing Messages into interfaces, which the receiving component drains when it is next stepped. That decoupling is what makes the hierarchy modular and back-pressured – an interface that is full simply refuses the push, and the sender tries again on its next tick.

Because all components step under one manager, latencies are consistent system-wide: the Logger measures every stage of a request in the *originating core's* clock, whichever component stamps it (Chapter 11). Each component carries its own m_clk_period, so a memory model may legitimately tick at a different rate from the cores; in the shipped configuration every period is 100 and one tick is one cycle everywhere.

Design note: a clock-period bug worth knowing about

The MCsim bridge once ticked the DRAM model once per *manager* step rather than once per LLC cycle – a hundred DRAM cycles per core cycle – so DRAM looked like a one-cycle device. The LLC's period is now passed into the bridge; any DRAM number produced before that fix is untrustworthy. The lesson generalises: a component with its own period must be stepped by the manager, never by whoever happens to call it.

3.3 Component map

A system is clusters of CPU cores, each with private caches and a direct interconnect, joined through configurable interconnects to a shared LLC and main memory (Figure 3.2). Table 3.2 names the class behind each role; Figure 3.3 shows how they relate.

3.4 The system as shipped

Running `Octopus_Simulator -s MultiCoreSystem` instantiates the C++ system class `MultiCoreSystem` and configures it from `configuration/SystemConfigurations/MultiCoreSystem`.

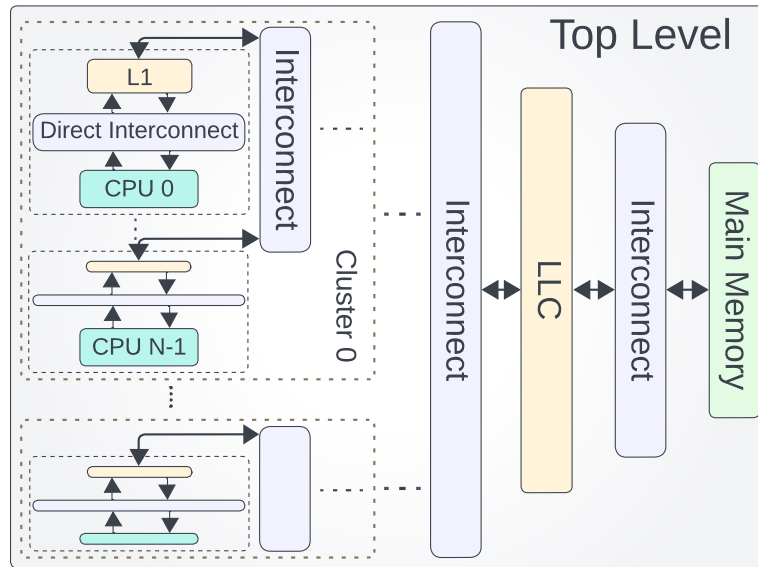


Figure 3.2. What a configured system can contain: clusters of cores with private caches on direct interconnects, further cache levels inside or across clusters, a shared last-level cache and main memory — all of it chosen by configuration, not hard-coded.

Table 3.2. Roles and the classes that play them.

role	class(es)	notes
clock	ClockManager, ClockedObj	drives cycleProcess() each tick
configuration	Configurable	hierarchical CSV + CLI parameters
core	CPU (trace), ExternalCPU (gem5)	issues Messages; the trace CPU has an out-of-order window of m_number_of_0o0_requests
transport	CommunicationInterface	pushMessage / peekMessage / popFrontMessage; bounded
message	Message	msg_id, addr, cycle, owner, source, data, kind
cache	BaseController → CacheController → CacheControllerExclusive, CacheController_End2End, CacheControllerDirectory	processing queue + handlers (Chapter 4)
coherence	CoherenceProtocolHandler → MSI / MESI / MOESI, their LLC and directory variants	driven by FSMReader (Chapter 6)
data	CacheDataHandler → array, MSHRs, write-back buffer, re- CacheDataHandler_COTS placement, port arbiter	
interconnect	InterconnectTopology + bus, mesh, NoC (Chapter 8) InterconnectController + Arbiter	
memory	MainMemoryController (fixed latency), MCsimInterface	the sink (Chapter 9)
monitoring	Logger, DebugPrint	Chapter 11

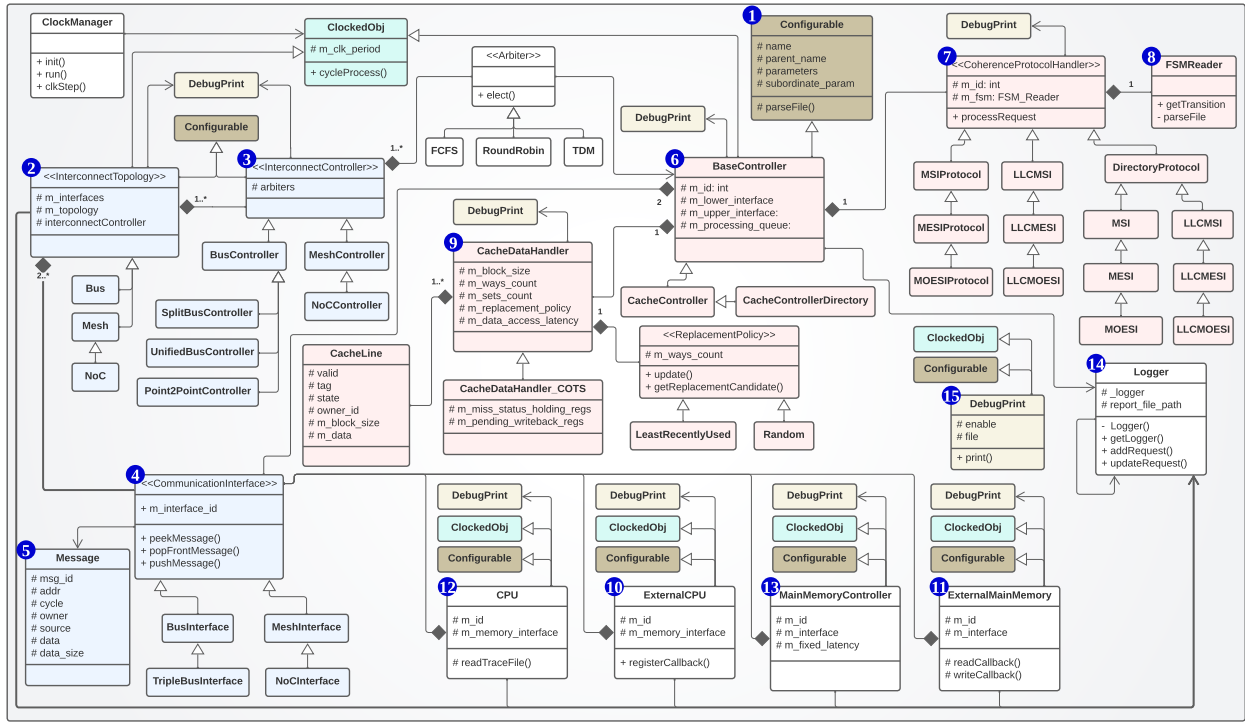


Figure 3.3. The class diagram. Cache-related classes in red, interconnect-related in blue, utilities in white.

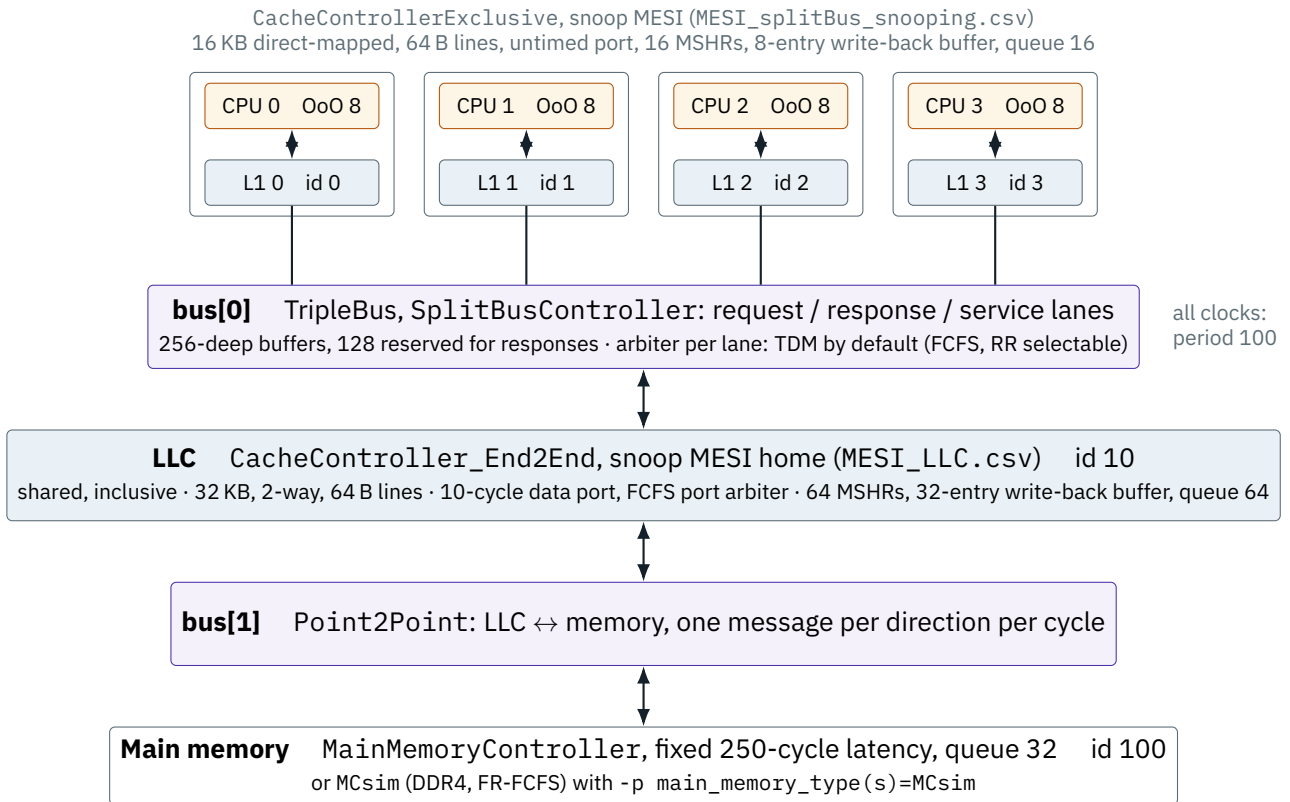


Figure 3.4. The system MultiCoreSystem.csv builds. Four trace-driven cores with private MESI L1s on a TripleBus, one shared inclusive LLC, a point-to-point memory bus and a fixed-latency main memory. Every number is a configuration key (Chapter A).

csv. Figure 3.4 is that system; it is the one every route, datapath and sequence figure in this manual is drawn on, and the one the tutorial runs first. The points that matter when reading a report:

- **Four cores** (`num_cores`), each a trace-driven CPU with an out-of-order window of eight outstanding requests, and each with a private L1 of `CacheControllerExclusive` running snoop MESI — 16 KB, direct-mapped, 64-byte lines, an untimed data port, 16 MSHRs, an 8-entry write-back buffer and a 16-deep processing queue.
- **bus[0]** is a `TripleBus` under a `SplitBusController`: three lanes — request, response, service — each with its own arbiter, TDM by default (`SplitBusController.csv`); 256-deep buffers with 128 entries reserved for in-flight responses so that the request lane stalls before responses can overflow.
- **The LLC** (id 10) is a `CacheController_End2End`: shared, inclusive, 32 KB, 2-way, 64-byte lines, a 10-cycle data port with an FCFS port arbiter, 64 MSHRs, a 32-entry write-back buffer and a 64-deep queue. Its MESI_LLC FSM is the home of the protocol.
- **bus[1]** is point-to-point between the LLC and memory (id 100).
- **Main memory** is a `MainMemoryController` with a fixed 250-cycle latency; `-p "main_memory_type(s swaps in cycle-accurate DDR4 (Chapter 9).`

Two sibling presets build the same topology with a different protocol: `MultiCoreSystem_Snoop.csv` (snoop MSI) and `MultiCoreSystem_Directory.csv` (directory MSI, pinned to an FCFS bus); `MultiCoreSystem_Mesh.csv` replaces the bus with a mesh (Chapter 8). A protocol is five coupled keys, which is why these are presets and not single `-p` overrides (Chapter 10).

Where in the code

```
src/ClockManager.cpp,    header/ClockManager.h    — the clock and
ClockedObj; src/Configurable.cpp — parameters; header/Interconnect/
CommunicationInterface.h — Message and the interfaces; src/
SystemConfigurations/MultiCoreSystem.cpp — the system class that wires
Figure 3.4; configuration/SystemConfigurations/MultiCoreSystem.csv — its
parameters; Octopus_Simulator.cpp, src/CacheSim.cpp — main and the run.
```

The cache controller

What you will find here. What every cache in Octopus is made of, whether it is a private L1 or the shared LLC: two interfaces, one processing queue, a protocol handler that reads a CSV state machine, and a data handler that owns the array, the MSHRs and the write-back buffer. How a message becomes a list of actions; what the controller admits and what it stalls; how the fields of a Message are decoded; and the one ordering rule at the bus interface that a snoop protocol's correctness rests on.

4.1 One class, every level

A cache controller is the same object at every level of the hierarchy. `BaseController` holds the machinery; `CacheController` adds the data array, the miss handling and the write-back path; `CacheControllerExclusive` (the shipped L1), `CacheController_End2End` (the shipped LLC) and `CacheControllerDirectory` (the directory home) specialise only what differs. What makes one instance an L1 and another the LLC is its position in the system and the FSM table it is given, not its class.

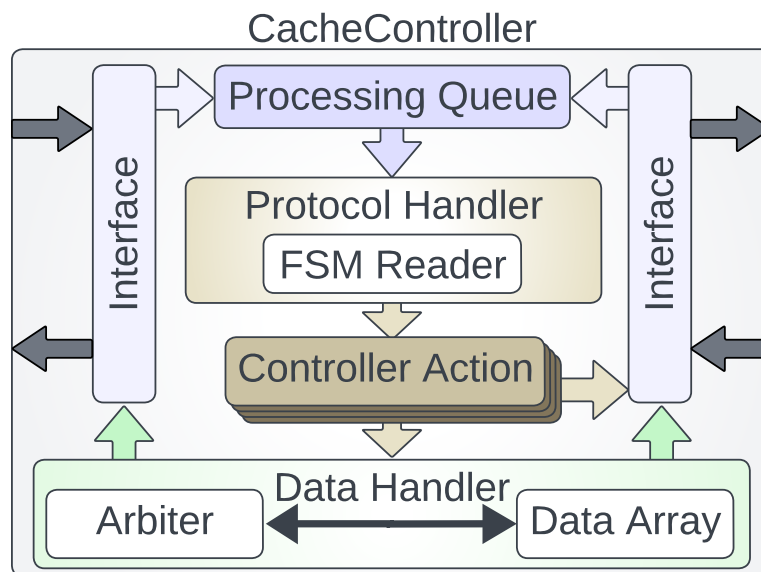


Figure 4.1. Inside a cache controller. Messages from both interfaces are serialised into one processing queue; the protocol handler turns each one into actions by looking the (state, event) pair up in the FSM table that the FSM reader parsed at start-up; the data handler executes those actions against the array, the MSHRs and the write-back buffer, arbitrating the data port.

Two interfaces. Every controller has a *lower* CommunicationInterface, facing the cores, and an *upper* one, facing the interconnect towards memory. Demand requests arrive from below; coherence requests, snooped transactions and data arrive from above; the controller's own requests leave through the upper interface. Interfaces are the only way in or out (Chapter 3): they are bounded queues, so a full interface pushes back on whoever is trying to fill it.

The processing queue. Messages from both interfaces are serialised into one queue ordered *first-ready, first-come-first-served* (FRFCFS_Buffer). A message is *ready* when the protocol handler judges that its line is in a state that can accept it; a message whose line is in a transient state waits without blocking the ones behind it that can proceed. The queue is bounded by `processing_queue_size`, and the bound applies to demand admission only: maintenance traffic the controller generates for itself, such as a write-back draining the buffer, is admitted with `force=true` so it can never be deadlocked by the demand it is making room for.

Per-line order. Among messages to the *same* line the queue keeps strict arrival order for demand requests and invalidations together; data messages are exempt, so a fill or a supply can always be processed and can never wait behind a stalled request. This per-line FCFS gate is what prevents an eviction's invalidation from leapfrogging an older GetS/GetM to the same line, and the Logger reports when a request was held by it (LLC Gate, Chapter 11).

4.2 From message to actions

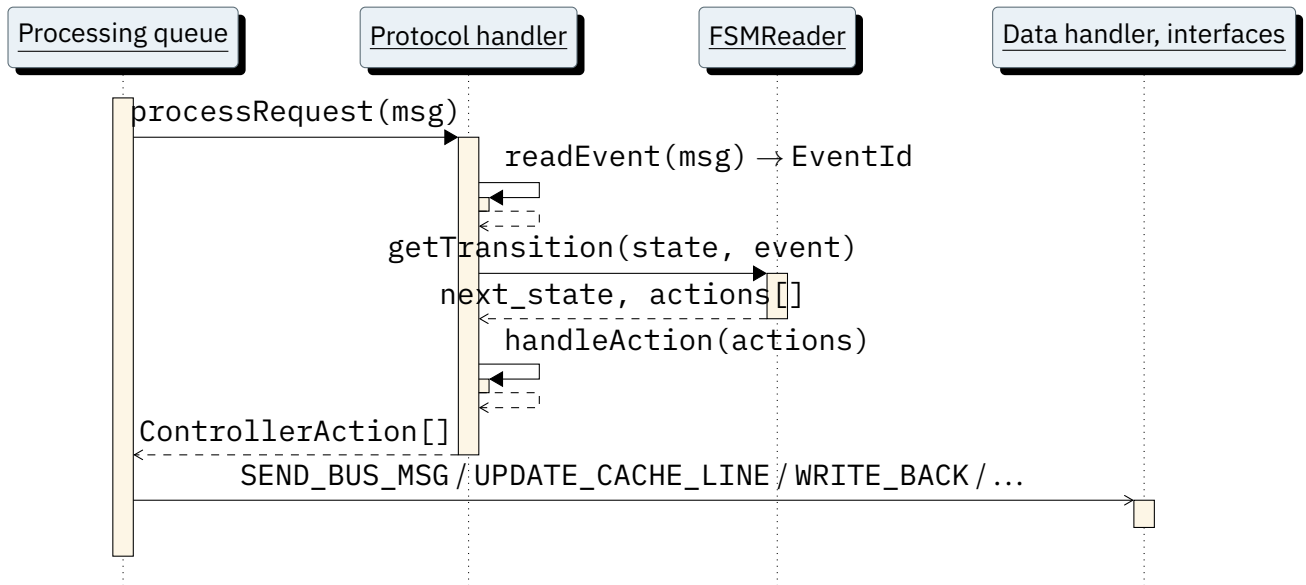


Figure 4.2. One message through the controller. The handler decodes the message into an event, looks up the transition for the line's current state, and hands the controller a list of protocol-agnostic actions; the controller does not know which protocol it is running.

The protocol handler (CoherenceProtocolHandler, specialised per family: MSIProtocol, MESIProtocol, MOESIProtocol, their LLC. . . and . . . Directory variants) does three things for every message (Figure 4.2):

1. **Decode the event.** `readEvent` maps the message to one column of the FSM table from its *source* and *contents*: a Load/Store from below; an Own_GetS/Own_GetM when the con-

troller sees its own request echoed back from the bus and `Other_GetS/Other_GetM` when it is another core's; `OwnData` when the message carries data for a line it is waiting on; `Invalidation` for an LLC back-invalidation; `Replacement` for the controller's own eviction.

2. **Look up the transition.** `FSMReader::getTransition(state, event)` returns the actions and next state the CSV table lists for that cell (Chapter 6).
3. **Translate to controller actions.** `handleAction` turns protocol actions such as `GetS`, `Data2Both` or `IssueInv` into `ControllerActions` the controller can execute without knowing the protocol: `SEND_BUS_MSG`, `UPDATE_CACHE_LINE`, `WRITE_BACK`, `HIT_Action`, `REMOVE_PENDING`, ...

A cell that reads `Fault/` means “this (state, event) must be unreachable”; the handler halts the run and reports it rather than mis-routing silently. That is what makes a protocol auditable: an incomplete table fails loudly, on the first trace that exercises the hole.

4.3 The data handler

`CacheDataHandler_COTS` owns everything that holds a line:

- **The array.** One array per controller, indexed by set, holding each line's tag, coherence bits and data together. Reading a line's *bits* (the tag compare, the state the FSM needs) is untimed and claims nothing. Reading or writing its *data* goes through a single port that is busy for `m_data_handler.m_data_access_latency` cycles per access; demand reads, eviction reads and fill or write-back writes contend for it through a round-robin arbiter (`m_data_access_arbiter`).
- **The MSHRs.** A miss allocates one entry and waits there, not in the array, until its data returns. `num_mshr` bounds the number of outstanding misses; `-1` removes the bound.
- **The write-back buffer.** A victim moves out of the array into the buffer, bits and data, the moment its way is needed, and leaves for memory later as a separate transaction. `m_data_handler.pwb_size` bounds it.
- **Replacement.** LRU, FIFO or random, chosen per controller; under way partitioning (Chapter 12) a core's fills install into, and evict from, its own ways only.

A line that currently sits in the MSHR or the write-back buffer is read without waiting for the port, because it is not in the array yet; it still restarts the port's busy timer, which is why such accesses cut in ahead of an access already in progress (the dotted bypass in Figure 5.1). Fill writes are not on the requester's critical path: the response is emitted from the fill message itself and the array write runs afterwards (Chapters 5 and 7).

4.4 Admission and back-pressure

A new demand miss is admitted only when the controller can finish it. `canAdmitRequest` stalls a demand request at the lower interface when the MSHRs are full or the write-back buffer has no headroom; `demandAdmissionBlocked` adds those two bounds to the processing-queue bound. Responses and the controller's own maintenance traffic are never subject to these checks, which is what keeps the system deadlock-free under finite buffers: a full queue of stalled requests can always drain, because the data they wait for is always admitted.

Design note: why the bounds are shaped this way

Finite queues and buffers once deadlocked the simulator: requests and responses shared one bounded queue, so a queue full of requests waiting for data could block the data they were waiting for. The fix is *response-safe admission*: only new demand is bounded, data and maintenance always get through, and each level carries its own finite caps. The default depths are in Chapter A.

4.5 The LLC's extra duties

The shipped LLC (CacheController_End2End) is *inclusive*: every line an L1 holds is also in the LLC. That gives it two duties an L1 does not have. When it evicts a line it must remove the L1 copies, which it does with the FSM action IssueInv: an INV broadcast on the TripleBus *service* lane that every L1 sees and that comes back to the LLC itself as Own_Invalidation to complete the eviction (Chapter 5). And it tracks, per line, which L1 currently owns it (SetOwner/ClearOwner), so that it can stay out of the way when an owner answers a request on the bus. The directory controller (CacheControllerDirectory) replaces snooping with a sharer list at the home and forwards requests point-to-point; Chapter 6 contrasts the two.

4.6 Message encoding

Every transaction is one Message object. The decoders in the protocol handlers read a small set of fields (Table 4.1); knowing them is what makes a Debugger trace or the raw event trace readable.

Table 4.1. The Message fields a decoder reads.

field	meaning
source	where the message entered the controller: LOWER_INTERCONNECT (from the cores' side), UPPER_INTERCONNECT (from the bus or memory side), SELF (a replacement the controller raised for itself)
data	NULL for a request or command; non-NULL for a data transfer
complementary_value	on a request, its type: 0 GetS, 1 GetM, 2 PutM, 10 INV (CPU→L1: 0 Load, 1 Store). On data it is <i>inherited</i> from the message the data was copied from: provenance, not a kind
owner	the requester on requests, supplies and fills; the evicting L1 on an eviction write-back; the LLC on an invalidation
to	destination interface ids; a response is delivered only to these, a broadcast request to everyone
msg_id	a demand request keeps one id from GetS/GetM through fill and response; an eviction write-back has its own id; a write-back forced by a back-invalidation carries the LLC eviction's id
kind	the producer's classification of the message (DEMAND, GETS, GETM, PUTM, INV, MEM_READ, EVICT, WB_DATA, WB_INV, SUPPLY, SUPPLY_DEFERRED, RESP, FILL, FILL_ROLLBACK, MEM_WRITE), set where it is created because the fields above cannot be read back into one

The bus MessageType passed to pushMessage (REQUEST, DATA_RESPONSE, SERVICE_REQUEST) selects the lane the message travels on; it is not stored in the message. Three things the same value means are worth remembering: cv = 2 is a PutM on a request, an eviction write-back at the LLC and an exclusive grant at a MESI L1; owner is the requester for requests, supplies and fills but the LLC for invalidations; and msg_id joins a demand request across its life, while an

invalidation-forced write-back is attributed to the LLC eviction, not to any core's request. The full producer-by-producer table is in docs/MessageEncoding.md.

4.7 Delivery order at the bus interface

A snoop protocol is correct only if every controller observes bus transactions in the *same* order. BusInterface therefore stamps each delivery into a controller's RX with one global sequence number, and TripleBusInterface hands out a service-lane message (an LLC back-invalidation) only when it was delivered *before* the request at the head of the request RX. Data responses are deliberately not ordered against requests, so that a full queue of stalled requests can always drain.

Design note: the invalidation that overtook a GetM

Giving the service lane unconditional priority once let a controller that was one message behind process an invalidation ahead of a GetM every other snooper had already seen; the LLC parked in a transient state with no exit. The global delivery sequence is the fix, and the reason the service lane is ordered against the request lane but data is not.

Where in the code

header/CacheControllers/BaseController.h, CacheController.h, CacheControllerExclusive.h, CacheController_End2End.h, CacheControllerDirectory.h and their src/CacheControllers/*.cpp — the controllers; src/CacheDataHandler_COTS.cpp — array, MSHRs, write-back buffer, port arbiter; header/FRFCFS_Buffer.h — the processing queue; src/Protocols/*.cpp — the handlers (readEvent, handleAction); header/Interconnect/CommunicationInterface.h — the message; src/Interconnect/BusInterface.cpp, TripleBusInterface.cpp — delivery order.

The last-level cache datapath

What you will find here. The shipped LLC drawn as a datapath, and the same drawing with one request's path highlighted four times: a read hit, a write hit, a read miss into a free way, and a miss that evicts. These five figures are the concrete instance of everything Chapter 4 describes, wired up the way the four-core system actually configures it, and they are where the latencies the simulator reports come from.

5.1 The datapath

Figure 5.1 follows the model rather than a textbook cache. The address splits into tag, set and offset, and the set indexes *one* array: in Octopus a line carries its coherence bits and its data together, so there is no separate tag array. That array is read two ways, and the difference matters for every latency the simulator reports. Reading a line's bits — the tag compare, the state the FSM needs — is **untimed** and claims nothing. Reading or writing its **data** goes through a single port, which demand reads, eviction reads and fill or write-back writes contend for through the round-robin arbiter, and which is then busy for A_LLC cycles. A line that currently sits in the MSHR or the write-back buffer is read without waiting for that port, because it is not in the array yet — the dotted bypass — but it still restarts the busy timer, which is why such accesses cut in ahead of one already in progress.

Control is the dashed amber layer: the line state and the decoded event enter the coherence FSM, whose next state is written back into the line and whose actions become port claims, buffer allocations and messages. A miss allocates in the MSHR and leaves on the memory bus; a victim moves to the write-back buffer and frees its way at once; and because the cache is inclusive, the FSM's IssueInv on an eviction invalidates the copy in whichever L1 holds the line — the red path, and the reason a task whose working set fits in its own private cache can still be disturbed by other cores (Chapter 12).

5.2 A read hit

Steps 4–6 — tag compare, FSM, sequencer — take no simulated time. What a hit pays for is the wait in the processing queue (1–3), the wait for the port (7), and then the A_LLC-cycle access itself (8–9). The figure draws the case where the LLC serves the data: the line is valid in the array and no L1 owns it (I or S in MESI_LLC.csv). The third case, a hit on a line an L1 holds in EorM, does not use the port at all: the owner answers the GetS on the bus with Data2Both, and the LLC only saves that copy (S_d → SaveData → S); on a GetM the owner hands the line straight to the requester and the LLC merely rewrites the owner bits.

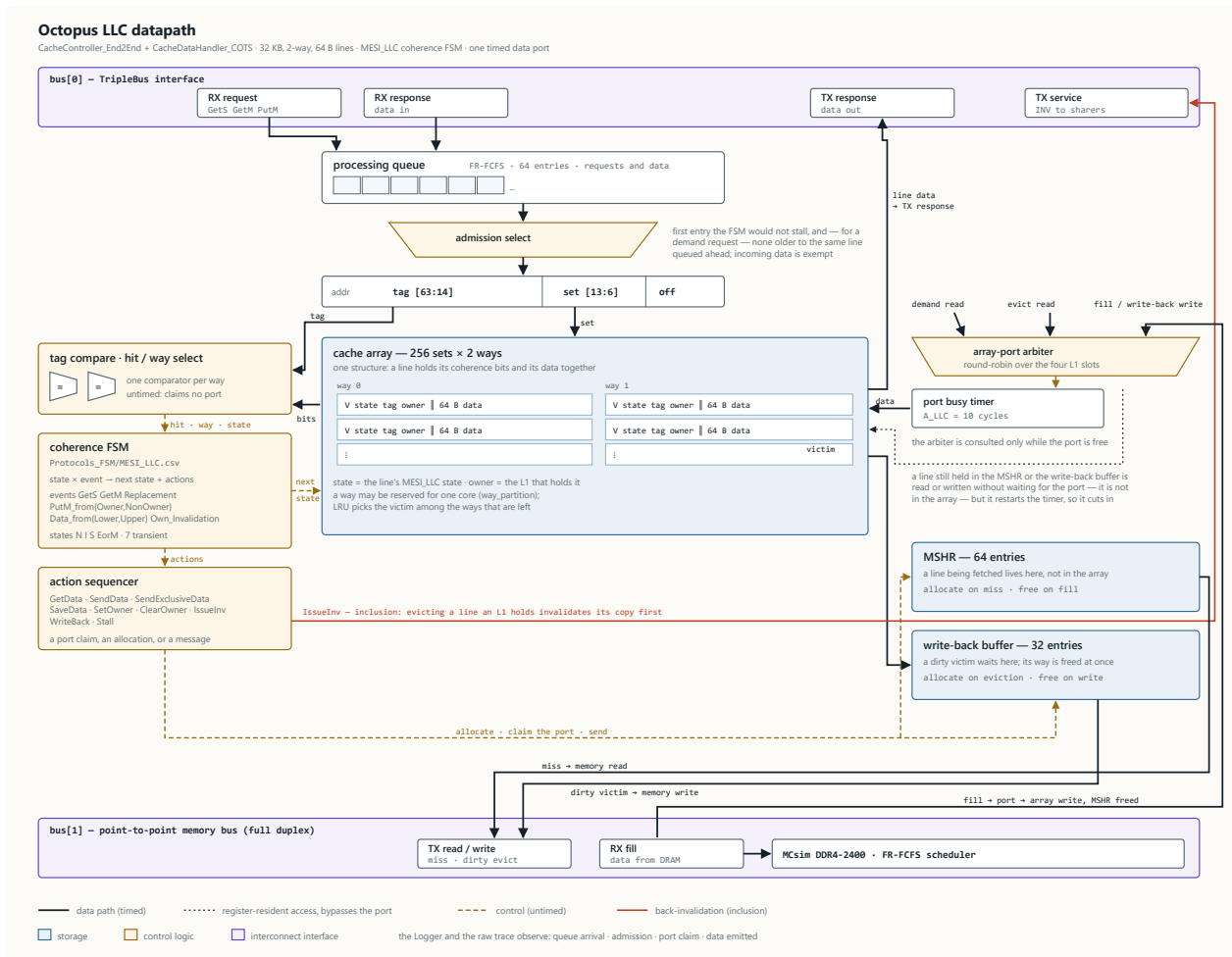


Figure 5.1. The shipped LLC as a datapath: CacheController_End2End with CacheDataHandler_COTS, 32 KB, 2-way, 64 B lines, the MESI_LLC FSM and one timed data port. Bus interfaces on top, control on the left, the single array in the middle read by an untimed bits path and a timed data port on the right, MSHR and write-back buffer beside it, the memory bus below.

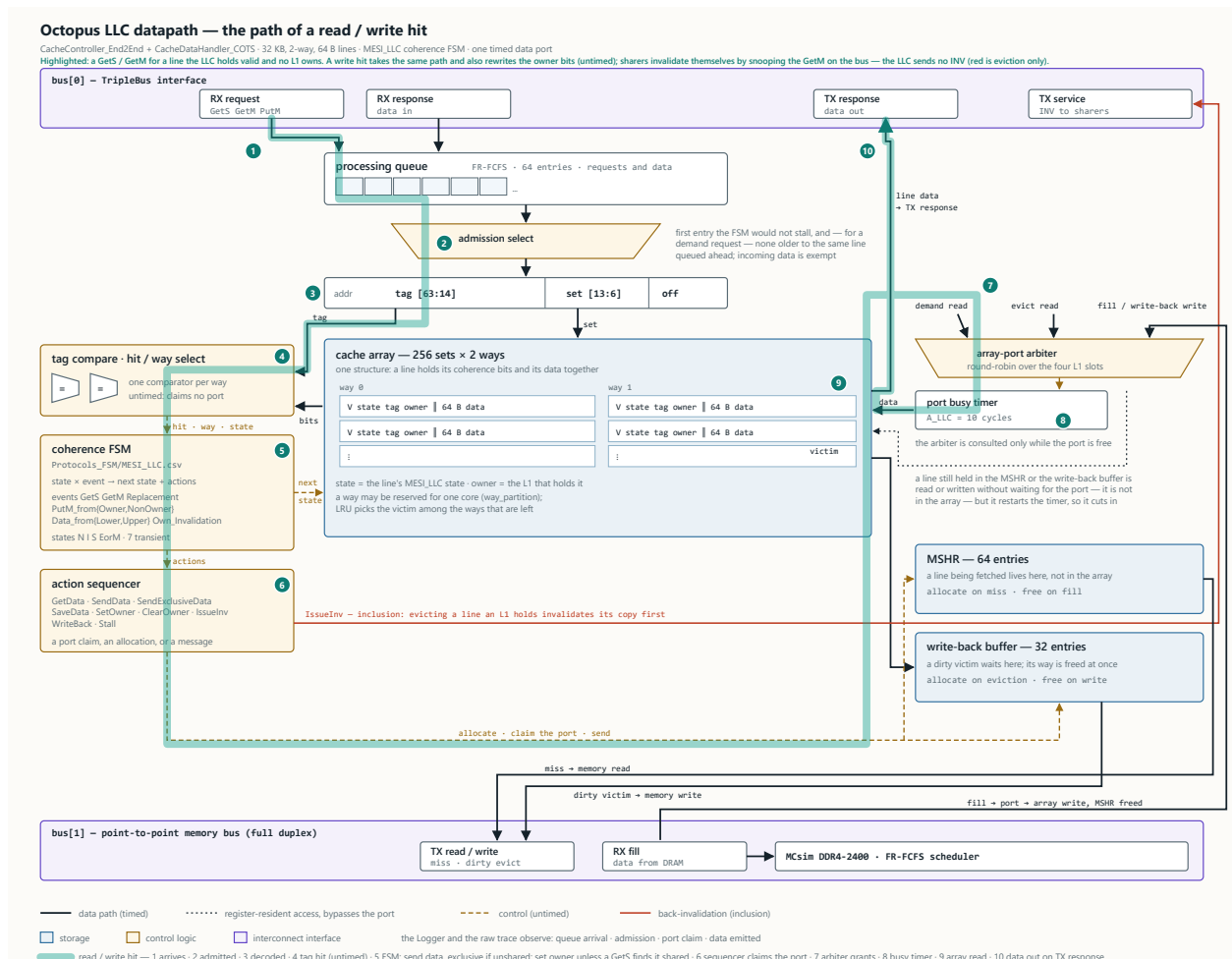


Figure 5.2. A read or write hit — a GetS or GetM that finds the line in a stable state — from the request's arrival on the bus to its data leaving on the response lane, in ten numbered steps.

5.3 A write hit

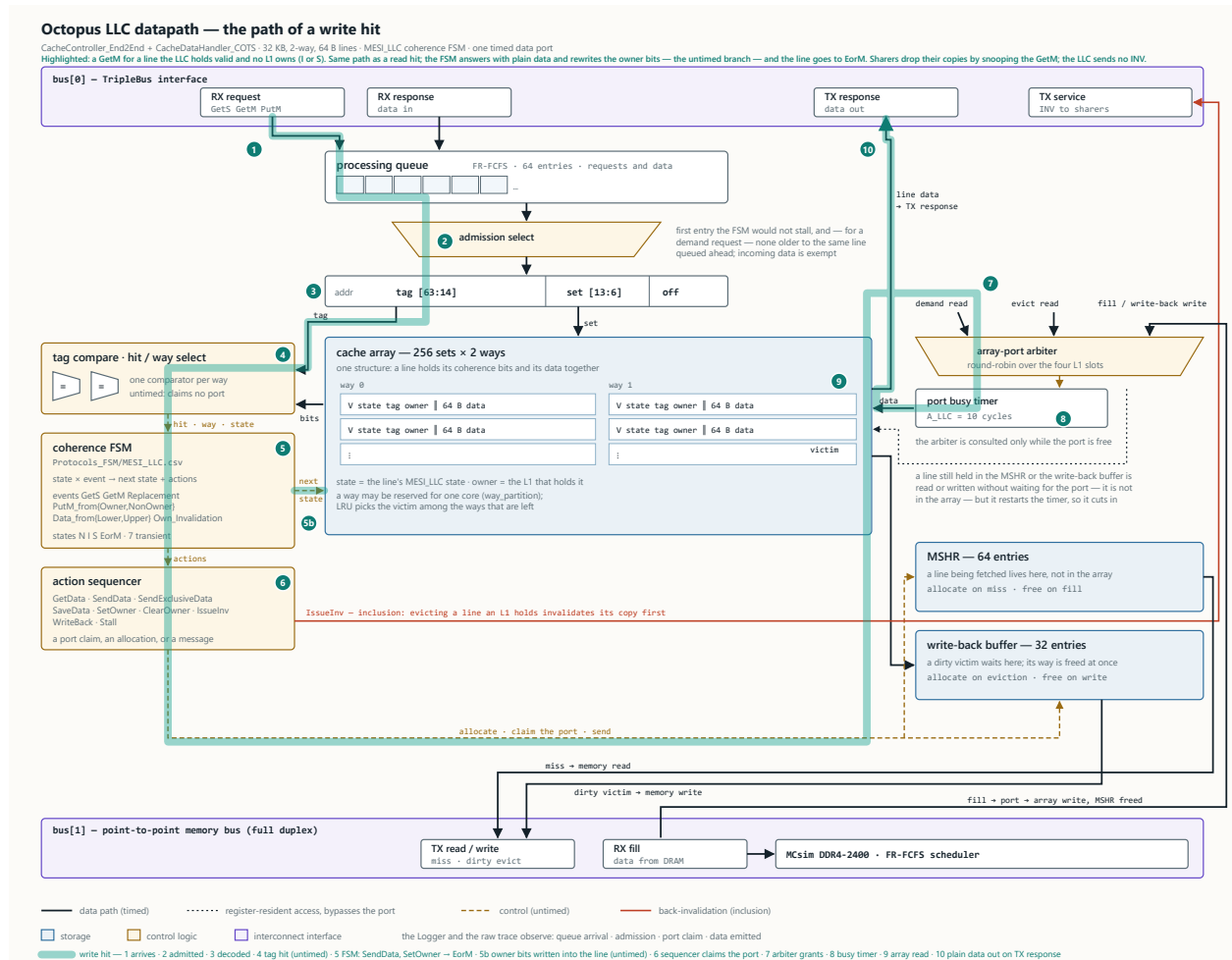


Figure 5.3. The write hit on its own. The path is the read hit's; what is new is the branch marked 5b — the FSM's SetOwner written back into the line's bits over the dashed control wire, untimed — and the plain-data response.

A write hit differs from a read hit in what the FSM does, not in the path: the GetM gets plain data where a GetS gets exclusive data, and the FSM also rewrites the owner bits. Nothing else changes — in particular the LLC sends **no** invalidation: on the snooping bus every sharer sees the GetM itself and drops its copy (S + Other_GetM → I), so IssueInv is reserved for evictions.

5.4 A read miss into a free way

The request takes the same first six steps and then diverges at the tag compare: the FSM's GetData allocates an MSHR entry, the read leaves on bus[1], and the line waits in the MSHR — not the array — until DRAM answers. When the fill is processed the response goes out on TX response at once, carrying the data from the fill message; the write into the free way — which contends for the port like any other array access and frees the MSHR — runs in parallel, off the requester's critical path. Because no victim is chosen, the write-back buffer and the red inclusion path are never touched. A write miss takes the identical route with a GetM in place of the GetS.

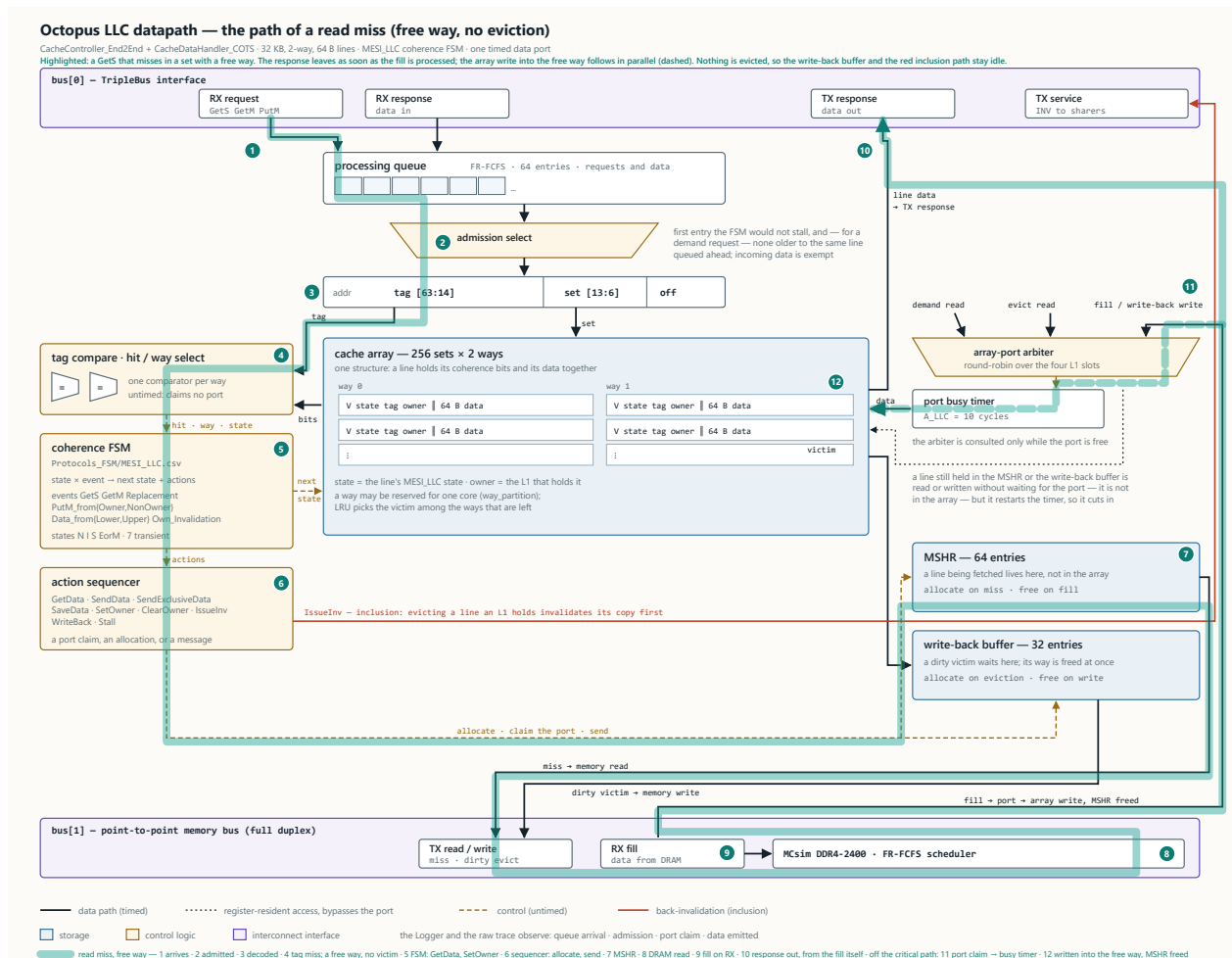


Figure 5.4. A read miss in the simplest case: the set still has a free way, so nothing is evicted. The request's own path in teal; the array write of the fill, off the critical path, dashed.

5.5 A miss that evicts

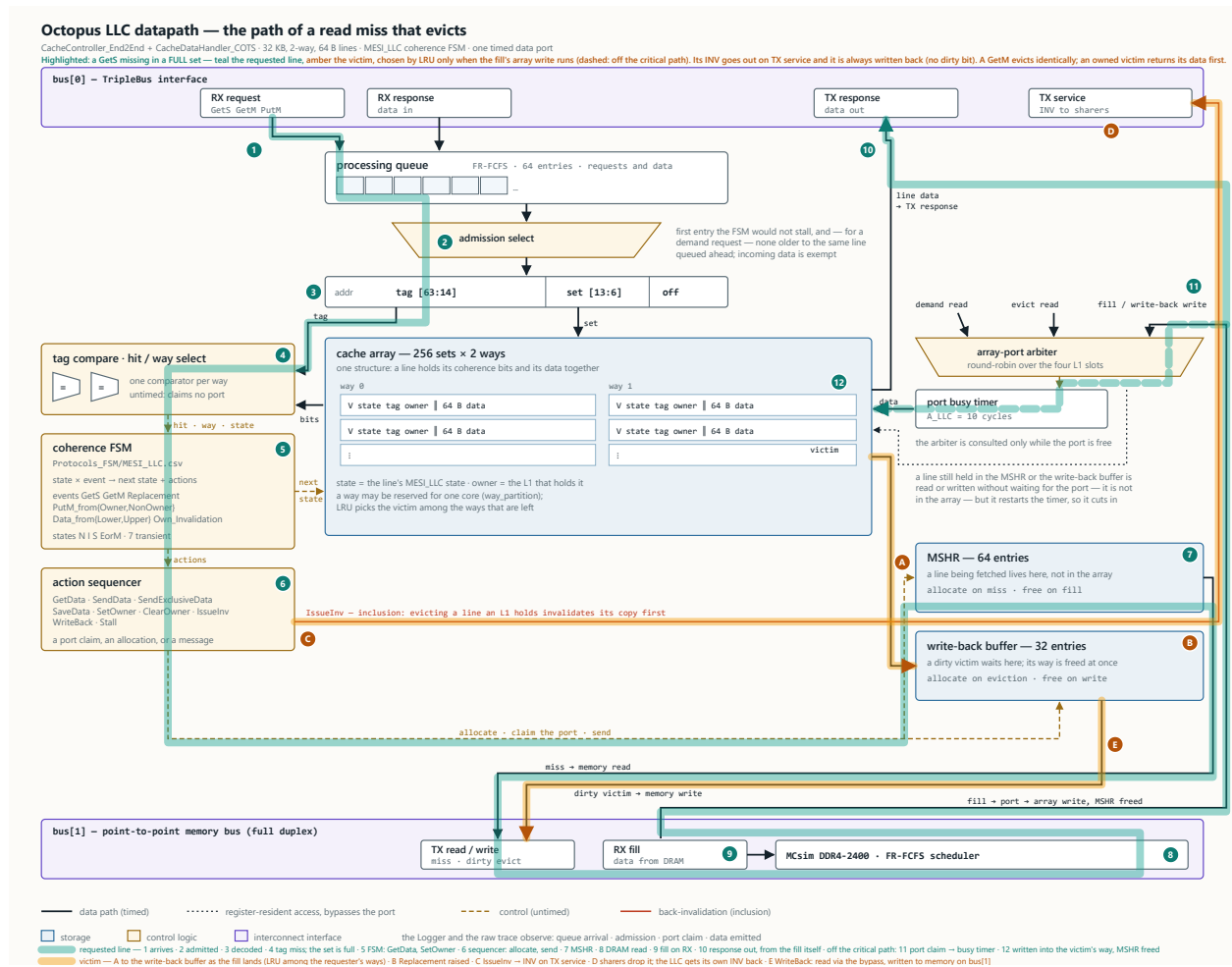


Figure 5.5. A miss into a *full* set: the requested line in teal, the evicted victim in amber. The victim is chosen when the fill is written, and its exit is a coherence transaction of its own.

Two things in Figure 5.5 are easy to get wrong from a textbook. First, **the victim is chosen late**: the miss allocates an MSHR and sends the read while the full set is left alone, and only when the fill's array write runs does the data handler pick the LRU line among the requester's allowed ways, move it — bits and data — into the write-back buffer, and put the fill in its way (moveLine2WB inside updateLineData). The requester's data leaves on TX response right then; everything the victim costs comes afterwards and lands on *other* traffic. Second, **the victim's exit is a coherence transaction**, not a buffer drain: a Replacement is raised for it and goes through the queue and the FSM like any request, IssueInv puts an INV on the service lane, every L1 that shares the line drops it, the LLC receives its own INV back, and only then does WriteBack read the line — from the buffer, over the dotted bypass — and send it to memory. There is no dirty bit in the model, so every victim is written back; the code marks the spot with a ToDo. A GetM miss evicts identically. A victim an L1 *owns* adds one leg: the owner answers the INV with its data, and that copy is what reaches memory.

Seen from the cores instead of from inside the LLC, that eviction is the whole story of inter-core interference through an inclusive LLC; Chapter 12 draws it as a sequence and Chapter 7 shows what it costs the victim's next request.

Where in the code

src/CacheControllers/CacheController_End2End.cpp — the LLC controller;
src/CacheDataHandler_COTS.cpp — updateLineData, moveLine2WB, the port
arbiter and the bypass; Protocols_FSM/MESI_LLC.csv — the FSM (Chapter B);
src/Protocols/LLCMESIProtocol.cpp — its handler; src/CacheControllers/
CacheController.cpp — sendInvalidationMessage, checkReplacements;
configuration/SystemConfigurations/MultiCoreSystem.csv — the
llc_controller.* keys that size it.

Chapter 6

The coherence engine: CSV finite-state machines

What you will find here. To be written.

Chapter 7

A request, end to end

What you will find here. One memory request followed from the core that issues it to the core that retires it, drawn on the system as shipped and on a time axis; the Logger stamp at every hop; and, from those stamps, the exact definition of every column of a `LatencyReport`. This is the chapter to have open the first time you read a report: a total tells you a core was slow, the columns tell you which shared resource made it slow, and this is what each column is.

7.1 The journey

A core issues a Message. Its L1 admits it, looks the line up and, on a miss, puts a GetS (read) or GetM (write) on the request lane of bus [0], where the arbiter elects it. Every other L1 snoops it; the LLC admits it into its queue. If the LLC has the line and no L1 owns it, the data is emitted when the array port is granted; if an L1 owns it, that L1 answers on the response lane instead; if the LLC does not have it, a read leaves on bus [1], DRAM serves it, the fill comes back to the front of the LLC's queue and the response is emitted from it. The response travels the response lane to the requesting L1, which fills and hands the data to the core.

Figures 7.1 and 7.2 are the same drawing with the two routes. Three things on them are worth saying in words. The other cores' L1s see every request on the request lane, and when one of them owns the line it answers on the response lane in the LLC's place, its stamps then standing in for the LLC's. The service lane carries the LLC's back-invalidations on an eviction and is on neither request's path. And a hit's L2 Access ends at the port *grant*: the data leaves then, and the read's A_LLC cycles on the port are paid by whoever waits for the port next — their L2 Access, or a fill's write.

7.2 The stamps

At each hop the component stamps a Logger checkpoint: an ENTER when it takes custody of the request, a SERVICE when it begins the actual work, an EXIT when it hands the request on. Every stamp names the component's *role* (CPU, L1, REQ_BUS, RESP_BUS, LLC, MEM_BUS, DRAM) and is taken in the originating core's clock, so a request's timeline is one ordered list of self-describing events and the columns are differences of *named* stamps, never inferences from position (Chapter 11 has the mechanism). Table 7.1 lists who stamps what.

7.3 The columns

Three things in Table 7.2 are easy to misread from the names alone.

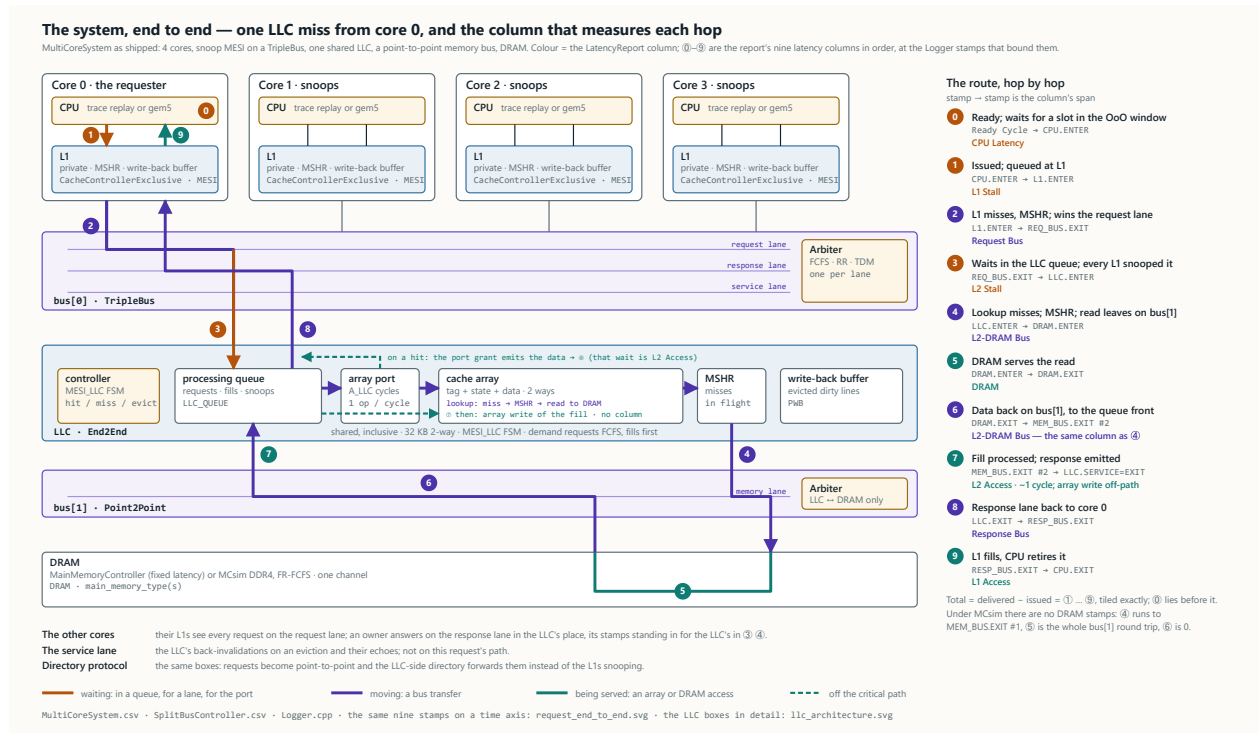


Figure 7.1. The system as shipped with one LLC *miss* from core 0 routed through it. Each hop carries the colour of the LatencyReport column that measures it (amber: waiting; purple: a bus transfer; teal: being served) and the number of the Logger stamp that bounds it, ①–⑩ being the report's nine latency columns in order; the side panel lists the stamp pair behind every column.

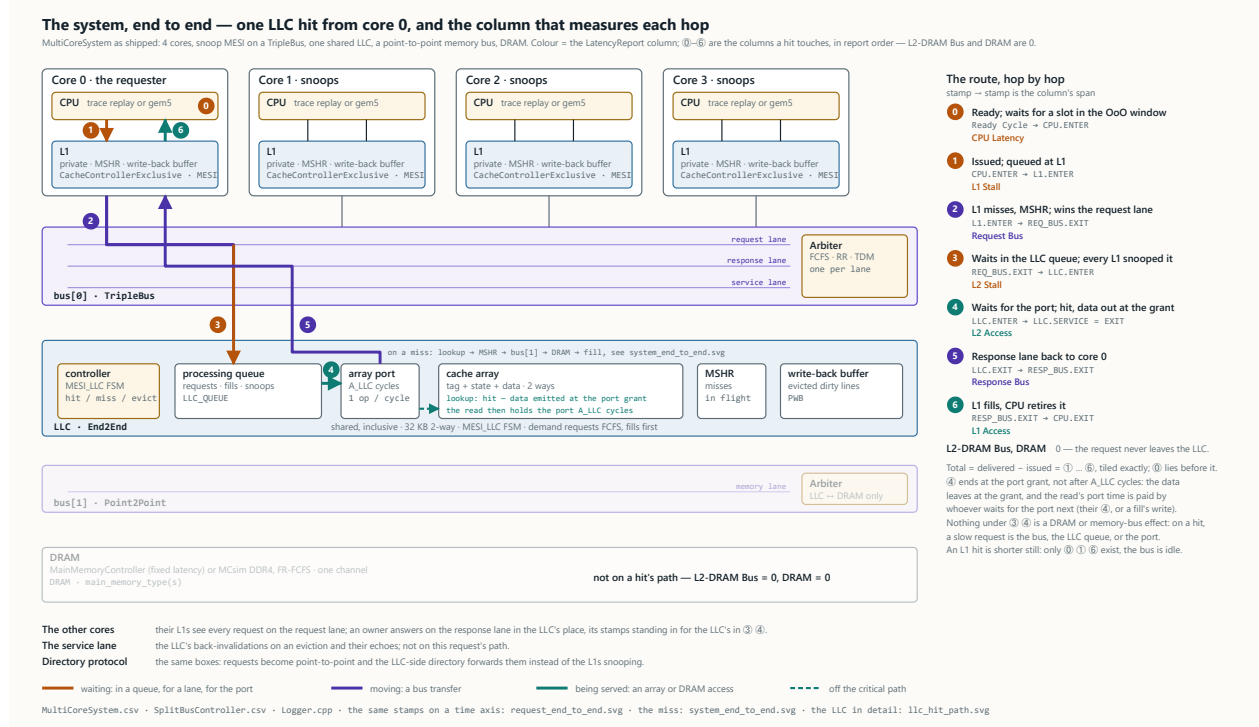


Figure 7.2. The same system with an LLC *hit* from core 0. The route stops at the LLC's array port, where the data is emitted at the grant; the memory bus and DRAM are not on the path, so L2-DRAM Bus and DRAM are 0.

Table 7.1. Where each Logger stamp is taken.

stamp	taken by
CPU.ENTER	CPU::addRequest: the request is issued (and bound to its core)
L1.ENTER	BaseController::processLogic when an L1 admits a request from below
REQ_BUS.EXIT	BusController::broadcast on bus[0]: the request is transmitted
LLC.ENTER	BaseController::processLogic when the LLC admits the bus request
MEM_BUS.EXIT	BusController on bus[1], once in each direction
DRAM.ENTER, DRAM.EXIT	MainMemoryController: request arrives, data leaves after m_memory_latency. MCsim stamps neither
LLC.SERVICE, LLC.EXIT	hitAction / removePendingAndRespond: port granted; response emitted
L1.SERVICE, L1.EXIT (before RESP_BUS.EXIT)	performWriteBack when an <i>owning</i> L1 supplies another core's request; the responder is then that L1, not the LLC
RESP_BUS.EXIT	BusController::send on bus[0]: the response is transmitted
CPU.EXIT	CPU::checkReceiveBuffer: delivered; triggers the report row

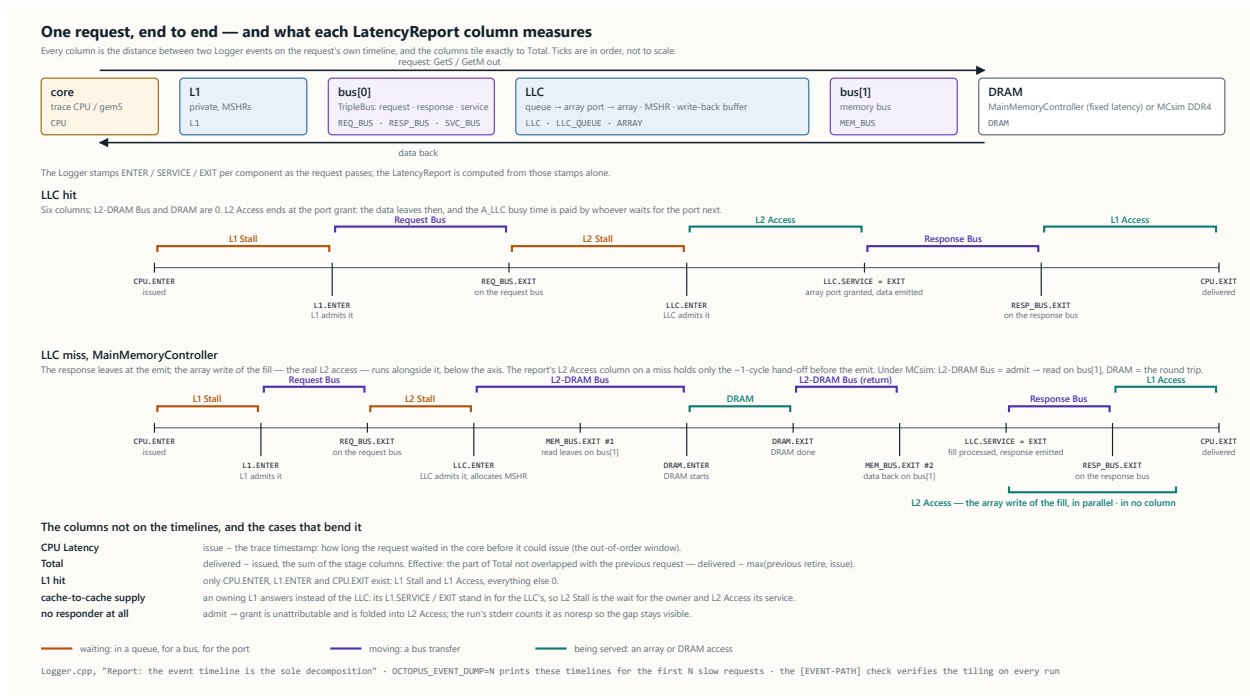
**Figure 7.3.** The same two routes on a time axis: an LLC hit and an LLC miss, with the nine LatencyReport columns bracketed between the Logger stamps that bound them. Ticks are in order, not to scale. Below the miss axis, the array write of the fill runs alongside the response and lands in no column.

Table 7.2. The LatencyReport columns, in report order, as differences of stamps. The nine latency columns tile exactly to Total; a stamp a request never reaches makes its column 0.

column	definition
CPU Latency	$\text{CPU}.\text{ENTER} - \text{Ready Cycle}$: the wait inside the core before the request could issue (the out-of-order window). Lies <i>before</i> Total
L1 Stall	$\text{L1}.\text{ENTER} - \text{CPU}.\text{ENTER}$
Request Bus	$\text{REQ_BUS}.\text{EXIT} - \text{L1}.\text{ENTER}$: the L1's miss handling and its wait for the request lane
L2 Stall	$\text{LLC}.\text{ENTER} - \text{REQ_BUS}.\text{EXIT}$: the wait in the LLC's queue
L2 Access	hit: $\text{LLC}.\text{SERVICE} - \text{LLC}.\text{ENTER}$, the wait for the array port; miss: $\text{LLC}.\text{SERVICE} - \text{MEM_BUS}.\text{EXIT}_2$, the fill's turn at the front of the queue, typically one cycle
Response Bus	$\text{RESP_BUS}.\text{EXIT} - \text{LLC}.\text{EXIT}$: the clean response-lane leg
L2-DRAM Bus	miss: $(\text{DRAM}.\text{ENTER} - \text{LLC}.\text{ENTER}) + (\text{MEM_BUS}.\text{EXIT}_2 - \text{DRAM}.\text{EXIT})$: one column over two hops, out and back
DRAM	miss: $\text{DRAM}.\text{EXIT} - \text{DRAM}.\text{ENTER}$
L1 Access	$\text{CPU}.\text{EXIT} - \max(\text{L1}.\text{ENTER}, \text{RESP_BUS}.\text{EXIT})$: an L1 hit's own service, or the fill and hand-off after the response-lane grant
Total	$\text{CPU}.\text{EXIT} - \text{CPU}.\text{ENTER}$ = the sum of the eight stage columns above (CPU Latency excluded)
Effective	$\text{CPU}.\text{EXIT} - \max(\text{previous request's finish}, \text{CPU}.\text{ENTER})$: the part of Total not overlapped with the previous request; its mean is the per-core average, its sum the true added execution time
Oldest	$\text{CPU}.\text{EXIT} - \text{the cycle this request became the oldest outstanding request of its core}$; 0 if it never was. At OoO= 1 it equals Total

L2 Access means two different waits. On a hit it ends at the array-port grant, not after A_LLC cycles: the data is emitted at the grant, and the port's busy time is paid by whoever waits for the port next. On a miss it is the fill's turn at the LLC: each cycle one message moves from the memory-side RX into the *front* of the processing queue, a data message is always ready, and every ready message is handled that same cycle — so typically a single cycle, plus any backlog in that one FIFO, and never a wait behind queued demand requests. The response is emitted the moment the fill is processed, carrying the data from the fill message; the array write claims the port afterwards, in parallel. That write is the real “L2 access” of a miss, Figure 7.3 draws it below the axis, and it lands in no report column (the ARRAY lane of the visualizer shows it).

L2-DRAM Bus and DRAM depend on the memory model. With MainMemoryController the DRAM stamps exist and L2-DRAM Bus is the two memory-bus legs either side of the DRAM service. MCsim stamps no DRAM events, so there L2-DRAM Bus becomes “LLC admit → read leaves on bus[1]” and DRAM becomes the whole bus[1] round trip, return leg included. Nothing of the round trip is lost, but the split moves.

A cache-to-cache supply substitutes the responder. When the LLC never emits because the line's owner answered on the bus, the last $\text{L1}.\text{SERVICE}/\text{L1}.\text{EXIT}$ strictly before $\text{RESP_BUS}.\text{EXIT}$ become the responder milestones, and the responder's SERVICE replaces $\text{LLC}.\text{ENTER}$ as the admission anchor — the owner snoops the request off the bus and may answer before the LLC admits it, so the LLC's admission is not on the data path. Then L2 Stall is the wait for the owner, L2 Access its service, and Response Bus still $\text{RESP_BUS}.\text{EXIT} - \text{responder EXIT}$. If no responder stamp exists at all, the whole admit-to-grant interval is folded into L2 Access so the columns keep tiling, and the run's stderr counts the request as noresp.

7.4 Checking a column

The columns tile to Total on every row by construction, and the simulator says so: the [EVENT - PATH] line on stderr at the end of a run reports how many requests tiled, how many did not, and how many had no responder. When a column looks wrong, OCTOPUS_EVENT_DUMP=N prints the raw timeline — every stamp with its role and cycle — of the first N requests whose Total exceeds 100 cycles or that crossed the memory bus; OCTOPUS_EVENT_DUMP_MIN=<cycles> raises the threshold to isolate one outlier row seen in a report. The seven *mechanism tracker* columns that follow Oldest in the report (the line’s state when the request arrived at the LLC, whether the per-line gate held it, how many younger responses and array accesses went ahead of it) attribute a stall to its cause; Chapter 11 defines them.

Where in the code

src/Logger.cpp — the stamps, finalizeEvents and the “Report: the event timeline is the sole decomposition” block that computes every column; header/Logger.h — the Role/Phase enums; docs/Logger.md — the same material as a reference; tutorial/00-setup-and-first-run — reading a first report.

Chapter 8

Interconnects and arbitration

What you will find here. To be written.

Chapter 9

Main memory

What you will find here. To be written.

Chapter 10

Configuration

What you will find here. To be written.

Chapter 11

Monitoring: Logger, Debugger, trace and visualizer

What you will find here. To be written.

Chapter 12

Periodic tasks and the interference demo

What you will find here. To be written.

Chapter 13

Extending Octopus

What you will find here. To be written.

Chapter 14

Integration modes: gem5 and the full-system stack

What you will find here. To be written.

Chapter 15

Hands-on tutorial

What you will find here. To be written.

Chapter 16

Evaluation

What you will find here. To be written.

Chapter A

Configuration key reference

Every key in `configuration/**/*`.csv as shipped, one table per file, generated by `docs/manual/tools/gen_config_tables.py` at build time. The type is the suffix the key carries in the CSV ((i) integer, (s) string, (vi)/(vs) lists); the value column is the shipped default. Chapter 10 explains how files extend each other and how a `-p` override reaches a key.

configuration/SystemConfigurations/MultiCoreSystem.csv

key	type	value	meaning
<code>workload_path</code>	string	<code>BMs/TestBM/</code>	Absolute path is recommended
<code>num_cores</code>	int	<code>4</code>	
<code>bus_type</code>	string list	<code>TripleBus,</code> <code>Bus</code>	Bus[0] (L1<->LLC) is TripleBus: carries the LLC back-invalidation on its service channel. Bus[1] is LLC<->memory snoop broadcast bus
<code>bus[0].controller_type</code>	string	<code>Split</code>	
<code>bus[0].m_lower_level_ids</code>	int list	<code>0, 1, 2, 3</code>	LLC ID
<code>bus[0].upper_level_cache_ids</code>	int list	<code>10</code>	
<code>cache_controller_type</code>	string	<code>Cache</code> <code>Controller</code> <code>Exclusive</code>	L1 private caches: basic snoop controller
<code>llc_controller_type</code>	string	<code>Cache</code> <code>Controller_</code> <code>End2End</code>	LLC: End2End controller drives TripleBus service channel for back-invalidation on eviction
<code>bus[*].buffers_max_size</code>	int	<code>256</code>	Response/request buffer depth: usable + in-flight reserve (sized to MSHR bound)
<code>bus[0].response_reserve</code>	int	<code>128</code>	Snoop bus in-flight response reserve: request bus stalls when response buffer free space falls to this (bounds response-bus back-pressure without overflow or controller gating)
<code>bus[1].m_lower_level_ids</code>	int list	<code>10</code>	LLC ID
<code>bus[1].upper_level_cache_ids</code>	int list	<code>100</code>	Main memory ID
<code>bus[1].controller_type</code>	string	<code>Point2Point</code>	
<code>bus[1].interconnect_controller.candidates_id</code>	int list	<code>0, 1, 2, 3,</code> <code>10</code>	Candidates are the l1 ids and llc id
<code>cache_controller[*].m_shared_memory_id</code>	int	<code>10</code>	

key	type	value	meaning
cache_controller[*].protocol_type	string	SNOOP_MESI	
cache_controller[*].fsm_filename	string	MESI_split Bus_snooping	
cache_controller[*].num_mshr	int	16	L1 MSHR depth (max concurrent outstanding misses). -1 = unbounded
cache_controller[*].m_data_handler.pwb_size	int	8	L1 write-back buffer depth. -1 = unbounded
cache_controller[*].processing_queue_size	int	16	L1 processing-queue depth (bounds demand admission). -1 = unbounded
cache_controller[0].m_id	int	0	
cache_controller[1].m_id	int	1	
cache_controller[2].m_id	int	2	
cache_controller[3].m_id	int	3	
llc_controller.m_id	int	10	LLC
llc_controller.m_shared_memory_id	int	100	Id of the main memory
llc_controller.protocol_type	string	SNOOP_LLC_MESI	
llc_controller.fsm_filename	string	MESI_LLC	
llc_controller.m_data_handler.cache_size	int	32768	
llc_controller.m_data_handler.m_ways_count	int	2	
llc_controller.m_data_handler.m_data_access_latency	int	10	
llc_controller.num_mshr	int	64	LLC MSHR depth: larger than L1 (fans in all cores). -1 = unbounded
llc_controller.m_data_handler.pwb_size	int	32	LLC write-back buffer depth: larger than L1. -1 = unbounded
llc_controller.processing_queue_size	int	64	LLC processing-queue depth: larger than L1 (holds cache-to-cache forwarding concurrency). -1 = unbounded
llc_controller.arbiter_type	string	RRArbiter	
llc_controller.arbiter_candidates_ids	int list	0, 1, 2, 3	Candidates are the l1 ids
m_main_memory.m_llc_id	int	10	
m_main_memory.m_id	int	100	
cache_controller[*].dprint.enable	int	0	
llc_controller.dprint.enable	int	0	
cache_controller[*].dprint.target	string	BMs/test DPrint/dprint.csv	
llc_controller.dprint.target	string	BMs/test DPrint/dprint.csv	
cache_controller[*].dprint.addr_filter	string		filter in hexadecimal
llc_controller.dprint.addr_filter	string		filter in hexadecimal
cache_controller[*].dprint.cond_addr	string list		Acceptable Addresses
llc_controller.dprint.cond_addr	string list		Acceptable Addresses

key	type	value	meaning
main_memory_type	string	MCsim	
mcsim_scheduler	string	FRFCFS	

configuration/SystemConfigurations/MultiCoreSystem_Directory.csv

key	type	value	meaning
workload_path	string	BMs/TestBM/	Absolute path is recommended
num_cores	int	4	
bus_type	string list	TripleBus, Bus	Bus[0] is the interconnect between private caches and LLC and Bus[1] is between LLC and main memory
cache_controller_type	string	Cache Controller Directory	Type should match the name of the class
bus[*].buffers_max_size	int	32	
bus[1].m_lower_level_ids	int list	10	LLC ID
bus[1].upper_level_cache_ids	int list	100	Main memory ID
bus[1].controller_type	string	Point2Point	
bus[1].interconnect_controller.candidates_id	int list	0, 1, 2, 3, 10	Candidates are the l1 ids and llc id
bus[0].interconnect_controller.arbiter_type	string	FCFSArbiter	Pinned: every directory validation was done under FCFS. The shared SplitBusController default is TDM (for snoop) and directory+TDM starves one core on aifirf01/cacheb01 – an open bug rather than a config choice
cache_controller[*].m_shared_memory_id	int	10	
cache_controller[*].protocol_type	string	DIRECTORY_MSI	
cache_controller[*].fsm_filename	string	MSI_directory	
cache_controller[*].num_mshr	int	16	L1 MSHR depth (max concurrent outstanding misses). -1 = unbounded
cache_controller[*].m_data_handler.pwb_size	int	8	L1 write-back buffer depth. -1 = unbounded
cache_controller[*].processing_queue_size	int	16	L1 processing-queue depth (bounds demand admission). -1 = unbounded
cache_controller[0].m_id	int	0	
cache_controller[1].m_id	int	1	
cache_controller[2].m_id	int	2	
cache_controller[3].m_id	int	3	
llc_controller.m_id	int	10	LLC
llc_controller.m_shared_memory_id	int	100	Id of the main memory
llc_controller.protocol_type	string	DIRECTORY_LLC_MSI	
llc_controller.fsm_filename	string	MSI_LLC_directory	
llc_controller.m_data_handler.cache_size	int	32768	
llc_controller.m_data_handler.m_ways_count	int	2	

key	type	value	meaning
llc_controller.m_data_handler.m_data_access_latency	int	10	
llc_controller.num_mshr	int	64	LLC MSHR depth: larger than L1 (fans in all cores). -1 = unbounded
llc_controller.m_data_handler.pwb_size	int	32	LLC write-back buffer depth: larger than L1. -1 = unbounded
llc_controller.processing_queue_size	int	32	LLC processing-queue depth: larger than L1. -1 = unbounded
llc_controller.arbiter_type	string	FCFSArbiter	
llc_controller.arbiter_candidates_ids	int list	0, 1, 2, 3	Candidates are the l1 ids
m_main_memory.m_llc_id	int	10	
m_main_memory.m_id	int	100	
cache_controller[*].dprint.enable	int	0	
llc_controller.dprint.enable	int	0	
cache_controller[*].dprint.target	string	BMs/test DPrint/dprint.csv	
llc_controller.dprint.target	string	BMs/test DPrint/dprint.csv	
cache_controller[*].dprint.addr_filter	string		filter in hexadecimal
llc_controller.dprint.addr_filter	string		filter in hexadecimal
cache_controller[*].dprint.cond_addr	string list		Acceptable Addresses
llc_controller.dprint.cond_addr	string list		Acceptable Addresses

configuration/SystemConfigurations/MultiCoreSystem_Mesh.csv

key	type	value	meaning
workload_path	string	BMs/TestBM/	Absolute path is recommended
num_cores	int	4	
cache_controller_type	string	Cache Controller Directory	Type should match the name of the class
interconnect_type	string	NoC	
interconnect.buffers_max_size	int	32	
bus.buffers_max_size	int	32	
bus.m_lower_level_ids	int list	10	LLC ID
bus.upper_level_cache_ids	int list	100	Main memory ID
bus.controller_type	string	Point2Point	
bus.interconnect_controller.candidates_id	int list	0, 1, 2, 3, 10	Candidates are the l1 ids and llc id
cache_controller[*].m_shared_memory_id	int	10	
cache_controller[*].protocol_type	string	DIRECTORY_ MSI	

key	type	value	meaning
cache_controller[*].fsm_filename	string	MSI_directory	
cache_controller[0].m_id	int	0	
cache_controller[1].m_id	int	1	
cache_controller[2].m_id	int	2	
cache_controller[3].m_id	int	3	
llc_controller.m_id	int	10	LLC
llc_controller.m_shared_memory_id	int	100	Id of the main memory
llc_controller.protocol_type	string	DIRECTORY_LLCSMSI	
llc_controller.fsm_filename	string	MSI_LLCSdirectory	
llc_controller.m_data_handler.cache_size	int	32768	
llc_controller.m_data_handler.m_ways_count	int	2	
llc_controller.m_data_handler.m_data_access_latency	int	10	
llc_controller.arbiter_type	string	RRArbiter	
llc_controller.arbiter_candidates_ids	int list	0, 1, 2, 3	Candidates are the l1 ids
m_main_memory.m_llc_id	int	10	
m_main_memory.m_id	int	100	
cache_controller[2].dprint.enable	int	0	
llc_controller.dprint.enable	int	0	
cache_controller[*].dprint.target	string		
llc_controller.dprint.target	string	BMS/testDPrint/dprint.csv	
cache_controller[*].dprint.addr_filter	string	0x filter in hexadecimalFFFFFFFFFFFFFFFFC0	
llc_controller.dprint.addr_filter	string	0x filter in hexadecimalFFFFFFFFFFFFFFFFC0	
cache_controller[*].dprint.cond_addr	string list		Acceptable Addresses
llc_controller.dprint.cond_addr	string list		Acceptable Addresses
cache_controller[2].dprint.cond_msg_id	int list	245788	

configuration/SystemConfigurations/MultiCoreSystem_Snoop.csv

key	type	value	meaning
workload_path	string	BMS/TestBM/	Absolute path is recommended
num_cores	int	4	
bus_type	string list	TripleBus, Bus	Bus[0] (L1<->LLC) is TripleBus: carries the LLC back-invalidation on its service channel. Bus[1] is LLC<->memory

key	type	value	meaning
bus[0].controller_type	string	Split	snoop broadcast bus
bus[0].m_lower_level_ids	int list	0, 1, 2, 3	LLC ID
bus[0].upper_level_cache_ids	int list	10	
cache_controller_type	string	Cache Controller	L1 private caches: basic snoop controller
llc_controller_type	string	Cache Controller_End2End	LLC: End2End controller drives TripleBus service channel for back-invalidation on eviction
bus[*].buffers_max_size	int	256	Response/request buffer depth: usable + in-flight reserve (sized to MSHR bound)
bus[0].response_reserve	int	128	Snoop bus in-flight response reserve: request bus stalls when response buffer free space falls to this (bounds response-bus back-pressure without overflow or controller gating)
bus[1].m_lower_level_ids	int list	10	LLC ID
bus[1].upper_level_cache_ids	int list	100	Main memory ID
bus[1].controller_type	string	Point2Point	Candidates are the l1 ids and llc id
bus[1].interconnect_controller.candidates_id	int list	0, 1, 2, 3, 10	
cache_controller[*].m_shared_memory_id	int	10	
cache_controller[*].protocol_type	string	SNOOP_MSI	
cache_controller[*].fsm_filename	string	MSI_split Bus_snooping	
cache_controller[*].num_mshr	int	16	L1 MSHR depth (max concurrent outstanding misses). -1 = unbounded
cache_controller[*].m_data_handler.pwb_size	int	8	L1 write-back buffer depth. -1 = unbounded
cache_controller[*].processing_queue_size	int	16	L1 processing-queue depth (bounds demand admission). -1 = unbounded
cache_controller[0].m_id	int	0	
cache_controller[1].m_id	int	1	
cache_controller[2].m_id	int	2	
cache_controller[3].m_id	int	3	
llc_controller.m_id	int	10	LLC
llc_controller.m_shared_memory_id	int	100	Id of the main memory
llc_controller.protocol_type	string	SNOOP_LLC_MSI	
llc_controller.fsm_filename	string	MSI_LLC	
llc_controller.m_data_handler.cache_size	int	32768	
llc_controller.m_data_handler.m_ways_count	int	2	
llc_controller.m_data_handler.m_data_access_latency	int	10	
llc_controller.num_mshr	int	64	LLC MSHR depth: larger than L1 (fans in all cores). -1 = unbounded
llc_controller.m_data_handler.pwb_size	int	32	LLC write-back buffer depth: larger than L1. -1 = unbounded

key	type	value	meaning
llc_controller.processing_queue_size	int	64	LLC processing-queue depth: larger than L1 (holds cache-to-cache forwarding concurrency). -1 = unbounded
llc_controller.arbiter_type	string	FCFSArbiter	
llc_controller.arbiter_candidates_ids	int list	0, 1, 2, 3	Candidates are the l1 ids
m_main_memory.m_llc_id	int	10	
m_main_memory.m_id	int	100	
cache_controller[*].dprint.enable	int	0	
llc_controller.dprint.enable	int	0	
cache_controller[*].dprint.target	string	BMs/test DPrint/dprint.csv	
llc_controller.dprint.target	string	BMs/test DPrint/dprint.csv	
cache_controller[*].dprint.addr_filter	string		filter in hexadecimal
llc_controller.dprint.addr_filter	string		filter in hexadecimal
cache_controller[*].dprint.cond_addr	string list		Acceptable Addresses
llc_controller.dprint.cond_addr	string list		Acceptable Addresses

configuration/CacheControllers/BaseController.csv

key	type	value	meaning
m_id	int	-1	
m_shared_memory_id	int	-1	
m_clk_period	int	100	Clock period in nano seconds
processing_queue_size	int	64	If size is -1 the queue will grow indefinitely
protocol_type	string	SN00P_MSI	Type should be written similar to the names in ProtocolType enum in Protocols/Protocols.h
fsm_filename	string	MSI_split Bus_snooping	Name should match the name of the file in Protocols_FSM directory
perfect_llc	int	0	1 = this controller never issues to memory above it (every access hits

configuration/CacheControllers/CacheController.csv

Extends BaseController; every key below overrides or adds to it.

key	type	value	meaning
arbiter_type	string	NULL	Arbiters are Null, RR, FCFS, and TDM
arbiter_candidates_ids	int list	0, 1, 2, 3	

key	type	value	meaning
num_mshr	int	16	Max concurrent outstanding misses (MSHR depth). -1 = unbounded

configuration/CacheControllers/CacheControllerDirectory.csv

Extends CacheController; every key below overrides or adds to it.

key	type	value	meaning
processing_queue_size	int	32	Directory based protocols require larger sizes than snooping. A good practice is to set it to num_cores*max_mem_req

configuration/CacheControllers/CacheControllerExclusive.csv

Extends CacheController; every key below overrides or adds to it.

key	type	value	meaning

configuration/CacheControllers/CacheController_End2End.csv

Extends CacheController; every key below overrides or adds to it.

key	type	value	meaning

configuration/Interconnect/Bus.csv

key	type	value	meaning
m_lower_level_ids	int list	0, 1, 2, 3	
upper_level_cache_ids	int list	10	
controller_type	string	Unified	
buffers_max_size	int	16	
m_clk_period	int	50	Clk period in nano seconds

configuration/Interconnect/BusController.csv

key	type	value	meaning
m_request_latency	int	2	
m_response_latency	int	5	
arbiter_type	string	RRArbiter	Type should match the name of the arbiter class

configuration/Interconnect/DirectInterconnect.csv

key	type	value	meaning
buffers_max_size	int	16	Clk period in nano seconds
m_clk_period	int	50	

configuration/Interconnect/Mesh.csv

key	type	value	meaning
component_ids	int list	0, 1, 2, 3, 10	Clk period in nano seconds
buffers_max_size	int	16	
m_clk_period	int	50	first entry is the node id and the other entries are what the node is connected to
connection_matrix[0]	int list	0, 1, 2, 3, 10	
connection_matrix[1]	int list	1, 0, 2, 3, 10	
connection_matrix[2]	int list	2, 0, 1, 3, 10	
connection_matrix[3]	int list	3, 0, 1, 2, 10	
connection_matrix[4]	int list	10, 0, 1, 2, 3	first entry is the node id and the other entries are what the node is connected to

configuration/Interconnect/MeshController.csv

key	type	value	meaning
m_request_latency	int	2	Type should match the name of the arbiter class
m_data_latency	int	5	
arbiter_type	string	FCFSArbiter	

configuration/Interconnect/NoC.csv

Extends Mesh; every key below overrides or adds to it.

key	type	value	meaning
switch_ids	int list	200	Ids of the switches in the network first entry is the node id and the other entries are what the node is connected to
connection_matrix[0]	int list	0, 200, 10	

key	type	value	meaning
connection_matrix[1]	int list	1, 200, 10	first entry is the node id and the other entries are what the node is connected to
connection_matrix[2]	int list	2, 200, 10	first entry is the node id and the other entries are what the node is connected to
connection_matrix[3]	int list	3, 200, 10	first entry is the node id and the other entries are what the node is connected to
connection_matrix[4]	int list	10, 0, 1, 2, 3	first entry is the node id and the other entries are what the node is connected to
connection_matrix[5]	int list	200, 0, 1, 2, 3	first entry is the node id and the other entries are what the node is connected to

configuration/Interconnect/NoCController.csv

Extends MeshController; every key below overrides or adds to it.

key	type	value	meaning
-----	------	-------	---------

configuration/Interconnect/Point2PointController.csv

Extends BusController; every key below overrides or adds to it.

key	type	value	meaning
candidates_id	int list		

configuration/Interconnect/SplitBusController.csv

Extends BusController; every key below overrides or adds to it.

key	type	value	meaning
arbiter_type	string	FCFSArbiter	Type should match the name of the arbiter class
response_reserve	int	0	Response-buffer slots reserved for in-flight responses; request bus stalls when free space falls to this. 0 = stall at full.

configuration/Interconnect/TripleBus.csv

Extends Bus; every key below overrides or adds to it.

key	type	value	meaning
controller_type	string	Triple	

configuration/Interconnect/TripleBusController.csv

Extends SplitBusController; every key below overrides or adds to it.

key	type	value	meaning
-----	------	-------	---------

configuration/Interconnect/UnifiedBusController.csv

Extends BusController; every key below overrides or adds to it.

key	type	value	meaning
-----	------	-------	---------

configuration/CPU.csv

key	type	value	meaning
m_clk_period	int	100	
m_number_of_0o0_requests	int	8	

configuration/CacheDataHandler.csv

key	type	value	meaning
cache_size	int	16384	Cache size in bytes
m_block_size	int	64	Block size in bytes
m_ways_count	int	1	Number of ways. If the number is 1 this will make the cache a direct-mapped cache
m_data_access_latency	int	0	The latency of reading or writing data. This latency does not apply on reading or writing cache lines bits
replacement_policy_name	string	LRU	Choose out of LRU or RANDOM
way_partition	string		Per-core way reservation "cores:ways;..." with a-b ranges (e.g. 0:0;1-3:1). Empty = no partitioning. Unlisted cores share the unclaimed ways

configuration/CacheDataHandler_COTS.csv

Extends CacheDataHandler; every key below overrides or adds to it.

key	type	value	meaning
pwb_size	int	8	Write-back buffer depth (pending write-backs). -1 = unbounded

configuration/DebugPrint.csv

key	type	value	meaning
m_clk_period	int	100	
target	string		Write file path or leave it empty to print on the console
enable	int	0	0 or 1
print_preamble	int	1	0 or 1
print_name	int	1	0 or 1
print_clk	int	0	0 or 1
print_msg_id	int	1	0 or 1
print_addr	int	0	0 or 1
print_from	int	1	0 or 1
print_to	int	1	0 or 1
print_owner	int	1	0 or 1
addr_filter	string		filter in hexadecimal
cond_addr	string list		Acceptable Addresses
cond_msg_id	int list		Acceptable Ids
cond_from	int list		Acceptable Senders
cond_owner	int list		Acceptable Owners

configuration/MainMemoryController.csv

key	type	value	meaning
m_clk_period	int	100	
m_id	int	100	
m_llc_id	int	10	
m_memory_latency	int	250	Latency in cycles
processing_queue_size	int	32	Request buffer depth. -1 = unbounded (legacy behavior)

The shipped coherence FSMs

The transition tables of the protocol families that ship as complete presets (snoop MSI and MESI, directory MSI) and the snoop MOESI pair, generated from `Protocols_FSM/*.csv` by `docs/manual/tools/gen_fsm_tables.py`. Rows are states (**bold** = stable, *italic* = transient), columns are events, a cell is actions/next_state. Chapter 6 explains how to read and edit them.

B.1 MSI_splitBus_snooping

Table B.1. MSI_splitBus_snooping: the numbered lists the table below is written in.

events	actions	states
	0 Stall	0 I
	1 Hit	1 IS_ad
	2 GetS	2 IS_d
	3 GetM	3 IS_a
	4 PutM	4 IS_dI
	5 Data2Req	5 IM_ad
	6 Data2Both	6 IM_d
	7 SaveReq	7 IM_a
	8 Fault	8 IM_dI
		9 IM_dS
		10 IM_dSI
		11 S
		12 SM_ad
		13 SM_d
		14 SM_a
		15 SM_dI
		16 SM_dS
		17 SM_dSI
		18 M
		19 MI_a
		20 MII_a
		21 II_a

Table B.2. MSI_splitBus_snooping: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	Load	Store	Replacement	Own_GetS	Own_GetM	Own_PutM	Other_GetS	Other_GetM	Other_PutM	OwnData	Invalidation
I	GetS/IS_ad	GetM/IM_ad		Fault	Fault	Fault				Fault	
<i>IS_ad</i>	Stall/	Stall/	Stall/	IS_d	Fault	Fault				IS_a	
<i>IS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault		IS_dI		Hit/S	IS_dI
<i>IS_a</i>	Stall/	Stall/	Stall/	Hit/S	Fault	Fault			Fault	Fault	IS_ad
<i>IS_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/I	
<i>IM_ad</i>	Stall/	Stall/	Stall/	Fault	IM_d	Fault				IM_a	
<i>IM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/IM_dS	SaveReq/IM_dI		Hit/M	SaveReq/IM_dI
<i>IM_a</i>	Stall/	Stall/	Stall/	Fault	Hit/M	Fault				Fault	IM_ad
<i>IM_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I	
<i>IM_dS</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault		IM_dSI		Hit/Data2Both/S	IM_dSI
<i>IM_dSI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Both/I	
S	Hit/	GetM/SM_ad	I	Fault	Fault	Fault		I		Fault	I
<i>SM_ad</i>	Hit/	Stall/	Stall/	Fault	SM_d	Fault		IM_ad		SM_a	IM_ad
<i>SM_d</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/SM_dS	SaveReq/SM_dI		Hit/M	SaveReq/IM_dI
<i>SM_a</i>	Hit/	Stall/	Stall/	Fault	Hit/M	Fault		IM_a	Fault	Fault	IM_a
<i>SM_dI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I	
<i>SM_dS</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault		SM_dSI		Hit/Data2Both/S	SM_dSI
<i>SM_dSI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Both/I	
M	Hit/	Hit/	PutM/MI_a	Fault	Fault	Fault	Data2Both/S	Data2Req/I		Fault	Data2Req/I
<i>MI_a</i>	Hit/	Hit/	Stall/	Fault	Fault	Data2Req/I	Data2Both/II_a	Data2Req/II_a		Fault	MII_a
<i>MII_a</i>	Hit/	Hit/	Stall/	Fault	Fault	Data2Req/I				Fault	
<i>II_a</i>	Stall/	Stall/	Stall/	Fault	Fault	I				Fault	

B.2 MSI_LLC

Table B.3. MSI_LLC: the numbered lists the table below is written in.

events	actions	states
0 GetS	0 Stall	0 N
1 GetM	1 GetData	1 NIorS_d
2 Replacement	2 SendData	2 NM_d
3 PutM_fromOwner	3 SaveData	3 IorS
4 PutM_fromNonOwner	4 SetOwner	4 M
5 Data_fromLowerInterface	5 ClearOwner	5 IorS_d
6 Data_fromUpperInterface	6 IssueInv	6 IorS_a
7 Own_Invalidation	7 WriteBack	7 N_d
	8 Fault	8 N_a

Table B.4. MSI_LLC: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	GetS	GetM	Replacement	PutM_fromOwner	PutM_fromNonOwner	Data_fromLowerInterface	Data_fromUpperInterface	Own_Invalidation
N	GetData/N_IorS_d	GetData/SetOwner/NM_d		Fault		Fault	Fault	Fault
<i>N_IorS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendData/IorS	Fault
<i>NM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendData/M	Fault
IorS	SendData/	SendData/SetOwner/M	IssueInv/	Fault		Fault	Fault	WriteBack/N
M	ClearOwner/IorS_d	SetOwner/	IssueInv/	ClearOwner/IorS_d		SaveData/IorS_a	Fault	ClearOwner/N_d
<i>IorS_d</i>	Stall/	Stall/	Stall/	Stall/		SaveData/IorS	Fault	N_d
<i>IorS_a</i>	ClearOwner/IorS	Stall/	Stall/	ClearOwner/IorS		Fault	Fault	WriteBack/N_a
<i>N_d</i>	Stall/	Stall/	Stall/	Fault		WriteBack/N	Fault	Fault
<i>N_a</i>	ClearOwner/N	Stall/	Stall/	ClearOwner/N		Fault	Fault	Fault

B.3 MESI_splitBus_snooping

Table B.5. MESI_splitBus_snooping: the numbered lists the table below is written in.

events	actions	states
0 Load	0 Stall	0 I
1 Store	1 Hit	1 IS_ad
2 Replacement	2 GetS	2 IS_d
3 Own_GetS	3 GetM	3 IS_a
4 Own_GetM	4 PutM	4 IS_dI
5 Own_PutM	5 Data2Req	5 IM_ad
6 Other_GetS	6 Data2Both	6 IM_d
7 Other_GetM	7 SaveReq	7 IM_a
8 Other_PutM	8 Fault	8 IM_dI
9 OwnData	9 removeSavedReq	9 IM_dS
10 Invalidation		10 IM_dSI
11 OwnData_Exclusive		11 S
		12 SM_ad
		13 SM_d
		14 SM_a
		15 SM_dI
		16 SM_dS
		17 SM_dSI
		18 M
		19 MI_a
		20 II_a
		21 E
		22 IES_d
		23 IES_dI

Table B.6. MESI_splitBus_snooping: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	Load	Store	Replacement	Own_GetS	Own_GetM	Own_PutM	Other_GetS	Other_GetM	Other_PutM	OwnData	Invalidation	OwnData_Exclusive
I	GetS/IS_ad	GetM/IM_ad		Fault	Fault	Fault				Fault		Fault
<i>IS_ad</i>	Stall/	Stall/	Stall/	IS_d	Fault	Fault				IS_a		Fault
<i>IS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/IES_d	SaveReq/IS_dI		Hit/S	IS_dI	Hit/E
<i>IS_a</i>	Stall/	Stall/	Stall/	Hit/S	Fault	Fault			Fault	Fault	IS_ad	Fault
<i>IS_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/removeSavedReq/I		Hit/Data2Req/I
<i>IM_ad</i>	Stall/	Stall/	Stall/	Fault	IM_d	Fault				IM_a		Fault
<i>IM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/IM_dS	SaveReq/IM_dI		Hit/M	SaveReq/IM_dI	Fault
<i>IM_a</i>	Stall/	Stall/	Stall/	Fault	Hit/M	Fault				Fault	IM_ad	Fault
<i>IM_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault
<i>IM_dS</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault		IM_dSI		Hit/Data2Both/S	IM_dSI	Fault
<i>IM_dSI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Both/I		Fault
S	Hit/	GetM/SM_ad	I	Fault	Fault	Fault		I		Fault	I	Fault
<i>SM_ad</i>	Hit/	Stall/	Stall/	Fault	SM_d	Fault		IM_ad		SM_a	IM_ad	Fault
<i>SM_d</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/SM_dS	SaveReq/SM_dI		Hit/M	SaveReq/IM_dI	Fault
<i>SM_a</i>	Hit/	Stall/	Stall/	Fault	Hit/M	Fault		IM_a	Fault	Fault	IM_a	Fault
<i>SM_dI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault
<i>SM_dS</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault		SM_dSI		Hit/Data2Both/S	SM_dSI	Fault
<i>SM_dSI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Both/I		Fault
M	Hit/	Hit/	PutM/MI_a	Fault	Fault	Fault	Data2Both/S	Data2Req/I		Fault	Data2Req/I	Fault
<i>MI_a</i>	Hit/	Hit/	Stall/	Fault	Fault	Data2Req/I	Data2Both/II_a	Data2Req/II_a		Fault	Data2Req/II_a	Fault
<i>II_a</i>	Stall/	Stall/	Stall/	Fault	Fault	I				Fault		Fault
E	Hit/	Hit/M	PutM/MI_a	Fault	Fault	Fault	Data2Both/S	Data2Req/I		Fault	Data2Req/I	Fault
<i>IES_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault		IES_dI		Hit/removeSavedReq/S	IS_dI	Hit/Data2Both/S
<i>IES_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/removeSavedReq/S	IS_dI	Hit/Data2Both/I

B.4 MESI_LLC

Table B.7. MESI_LLC: the numbered lists the table below is written in.

events	actions	states
0 GetS	0 Stall	0 N
1 GetM	1 GetData	1 NE_d
2 Replacement	2 SendData	2 NM_d
3 PutM_fromOwner	3 SaveData	3 I
4 PutM_fromNonOwner	4 SetOwner	4 S
5 Data_fromLowerInterface	5 ClearOwner	5 EorM
6 Data_fromUpperInterface	6 IssueInv	6 IorS_a
7 Own_Invalidation	7 WriteBack	7 MN_d
	8 Fault	8 S_d
	9 SendExclusiveData	9 I_d
		10 N_a

Table B.8. MESI_LLC: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	GetS	GetM	Replacement	PutM_fromOwner	PutM_fromNonOwner	Data_fromLowerInterface	Data_fromUpperInterface	Own_Invalidation
N	GetData/SetOwner/NE_d	GetData/SetOwner/NM_d		Fault		Fault	Fault	SaveData/N
<i>NE_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendExclusiveData/EorM	Fault
<i>NM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendData/EorM	Fault
I	SendExclusiveData/SetOwner/EorM	SendData/SetOwner/EorM	IssueInv/	Fault		Fault	Fault	WriteBack/N
S	SendData/	SendData/SetOwner/EorM	IssueInv/	Fault		Fault	Fault	WriteBack/N
EorM	ClearOwner/S_d	SetOwner/	IssueInv/	ClearOwner/I_d		SaveData/IorS_a	Fault	ClearOwner/MN_d
<i>IorS_a</i>	ClearOwner/S	Stall/	Stall/	ClearOwner/I		Fault	Fault	WriteBack/N_a
<i>MN_d</i>	Stall/	Stall/	Stall/	Fault		WriteBack/N	Fault	SaveData/MN_d
<i>S_d</i>	Stall/	Stall/	Stall/	Fault		SaveData/S	Fault	MN_d
<i>I_d</i>	Stall/	Stall/	Stall/	Fault		SaveData/I	Fault	MN_d
<i>N_a</i>	ClearOwner/N	Stall/	Stall/	ClearOwner/N		Fault	Fault	Fault

B.5 MOESI_splitBus_snooping

Table B.9. MOESI_splitBus_snooping: the numbered lists the table below is written in.

events	actions	states
0 Load	0 Stall	0 I
1 Store	1 Hit	1 IS_ad
2 Replacement	2 GetS	2 IS_d
3 Own_GetS	3 GetM	3 IS_a
4 Own_GetM	4 PutM	4 IS_dI
5 Own_PutM	5 Data2Req	5 IM_ad
6 Other_GetS	6 Data2Both	6 IM_d
7 Other_GetM	7 SaveReq	7 IM_a
8 Other_PutM	8 Fault	8 IM_dI
9 OwnData	9 removeSavedReq	9 IM_d0
10 Invalidation		10 S
11 OwnData_Exclusive		11 SM_ad
		12 SM_d
		13 SM_a
		14 SM_dI
		15 SM_d0
		16 M
		17 MI_a
		18 II_a
		19 E
		20 O
		21 OM_a
		22 OI_a
		23 EI_a
		24 IES_d
		25 IES_dI
		26 EM_a
		27 IM_dOI
		28 SM_dOI

Table B.10. MOESI_splitBus_snooping: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	Load	Store	Replacement	Own_GetS	Own_GetM	Own_PutM	Other_GetS	Other_GetM	Other_PutM	OwnData	Invalidation	OwnData_Exclusive
I	GetS/IS_ad	GetM/IM_ad		Fault	Fault	Fault				Fault		Fault
<i>IS_ad</i>	Stall/	Stall/	Stall/	IS_d	Fault	Fault				IS_a		Fault
<i>IS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/IES_d	SaveReq/IS_dI		Hit/S	IS_dI	Hit/E
<i>IS_a</i>	Stall/	Stall/	Stall/	Hit/S	Fault	Fault			Fault	Fault	IS_ad	Fault
<i>IS_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/removeSavedReq/I		Hit/Data2Req/I
<i>IM_ad</i>	Stall/	Stall/	Stall/	Fault	IM_d	Fault				IM_a		Fault
<i>IM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/IM_dO	SaveReq/IM_dI		Hit/M	SaveReq/IM_dI	Fault
<i>IM_a</i>	Stall/	Stall/	Stall/	Fault	Hit/M	Fault				Fault	IM_ad	Fault
<i>IM_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault
<i>IM_dO</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/	SaveReq/IM_dOI		Hit/Data2Req/O	SaveReq/IM_dOI	Fault
S	Hit/	GetM/SM_ad	I	Fault	Fault	Fault		I		Fault	I	Fault
<i>SM_ad</i>	Hit/	Stall/	Stall/	Fault	SM_d	Fault		IM_ad		SM_a	IM_ad	Fault
<i>SM_d</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/SM_dO	SaveReq/SM_dI		Hit/M	SaveReq/IM_dI	Fault
<i>SM_a</i>	Hit/	Stall/	Stall/	Fault	Hit/M	Fault		IM_a	Fault	Fault	IM_a	Fault
<i>SM_dI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault
<i>SM_dO</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault	SaveReq/	SaveReq/SM_dOI		Hit/Data2Req/O	SaveReq/SM_dOI	Fault
M	Hit/	Hit/	PutM/MI_a	Fault	Fault	Fault	Data2Req/S	Data2Req/I		Fault	Data2Req/I	Fault
<i>MI_a</i>	Hit/	Hit/	Stall/	Fault	Fault	Data2Req/I	Data2Req/OI_a	Data2Req/II_a		Fault	Data2Req/II_a	Fault
<i>II_a</i>	Stall/	Stall/	Stall/	Fault	Fault	I				Fault		Fault
E	Hit/	GetM/EM_a	PutM/EI_a	Fault	Fault	Fault	Data2Req/S	Data2Req/I		Fault	I	Fault
O	Hit/	GetM/OM_a	PutM/OI_a	Fault	Fault	Fault	Data2Req/	Data2Req/I		Fault	Data2Req/I	Fault
<i>OM_a</i>	Hit/	Stall/	Stall/	Fault	Hit/M	Fault	Data2Req/	Data2Req/IM_ad		Fault	Data2Req/IM_ad	Fault
<i>OI_a</i>	Hit/	Stall/	Stall/	Fault	Fault	Data2Req/I	Data2Req/	Data2Req/II_a		Fault	Data2Req/II_a	Fault
<i>EI_a</i>	Hit/	Stall/	Stall/	Fault	Fault	I	Data2Req/	Data2Req/II_a		Fault	II_a	Fault
<i>IES_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault		IES_dI		Hit/removeSavedReq/S	IS_dI	Hit/Data2Req/S
<i>IES_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/removeSavedReq/S	IS_dI	Hit/Data2Req/I
<i>EM_a</i>	Stall/	Stall/	Stall/	Fault	Hit/M	Fault	Data2Req/SM_ad	Data2Req/IM_ad	Stall/	Fault	IM_ad	Fault
<i>IM_dOI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault
<i>SM_dOI</i>	Hit/	Stall/	Stall/	Fault	Fault	Fault				Hit/Data2Req/I		Fault

B.6 MOESI_LLC

Table B.11. MOESI_LLC: the numbered lists the table below is written in.

events	actions	states
0 GetS	0 Stall	0 N
1 GetM	1 GetData	1 NE_d
2 Replacement	2 SendData	2 NM_d
3 PutM_fromOwner	3 SaveData	3 I
4 PutM_fromNonOwner	4 SetOwner	4 S
5 Data_fromLowerInterface	5 ClearOwner	5 E
6 Data_fromUpperInterface	6 IssueInv	6 I_a
7 Own_Invalidation	7 WriteBack	7 MN_d
	8 Fault	8 S_d
	9 SendExclusiveData	9 I_d
		10 O
		11 S_a
		12 O_d
		13 M

Table B.12. MOESI_LLC: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	GetS	GetM	Replacement	PutM_fromOwner	PutM_fromNonOwner	Data_fromLowerInterface	Data_fromUpperInterface	Own_Invalidation
N	GetData/SetOwner/NE_d	GetData/SetOwner/NM_d		Fault		Fault	Fault	SaveData/N
<i>NE_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendExclusiveData/E	Fault
<i>NM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	SendData/M	Fault
I	SendExclusiveData/SetOwner/E	SendData/SetOwner/M	IssueInv/	Fault		Fault	Fault	WriteBack/N
S	SendData/	SendData/SetOwner/M	IssueInv/	Fault		Fault	Fault	WriteBack/N
E	ClearOwner/S	SetOwner/M	IssueInv/	ClearOwner/I		Fault	Fault	ClearOwner/WriteBack/N
<i>I_a</i>	Stall/	Stall/	Stall/	ClearOwner/I		Fault	Fault	Fault
<i>MN_d</i>	Stall/	Stall/	Stall/	Fault		WriteBack/N	Fault	SaveData/MN_d
<i>S_d</i>	Stall/	Stall/	Stall/	Fault		SaveData/S	Fault	MN_d
<i>I_d</i>	Stall/	Stall/	Stall/	Fault		SaveData/I	Fault	MN_d
O		SetOwner/M	IssueInv/	S_d		SaveData/S	Fault	ClearOwner/MN_d
<i>S_a</i>	Stall/	Stall/	Stall/	ClearOwner/S		Fault	Fault	Fault
<i>O_d</i>	Stall/	Stall/	Stall/	Fault		SaveData/O	Fault	MN_d
M	0	SetOwner/M	IssueInv/	ClearOwner/I_d		SaveData/I_a	Fault	ClearOwner/MN_d

B.7 MSI_directory

Table B.13. MSI_directory: the numbered lists the table below is written in.

events	actions	states
0 Load	0 Stall	0 I
1 Store	1 Hit	1 IS_d
2 Replacement	2 GetS	2 IS_dI
3 Fwd_GetS	3 GetM	3 IM_ad
4 Fwd_GetM	4 PutS	4 IM_a
5 Inv	5 PutM_Data	5 IM_aS
6 Put_Ack	6 Data2Both	6 IM_aSI
7 Data_zeroAck	7 Data2Req	7 IM_aI
8 Data_nonZeroAck	8 InvAck2Req	8 S
9 DataFromOwner	9 InvAck_Data	9 SM_ad
10 InvAck	10 Ack_dec	10 SM_a
11 lastInvAck	11 AckNum_set	11 M
	12 SaveReq	12 MI_a
	13 Fault	13 SI_a
		14 II_a

Table B.14. MSI_directory: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	Load	Store	Replacement	Fwd_getS	Fwd_getM	Inv	Put_Ack	Data_zeroAck	Data_nonZeroAck	DataFromOwner	InvAck	LastInvAck
I	GetS/IS_d	GetM/IM_ad		Fault	Fault	Fault	Fault	Fault	Fault	Fault	Fault	Fault
<i>IS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	InvAck2Req/IS_dI	Fault	Hit/S	Fault	Hit/S	Fault	Fault
<i>IS_dI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Hit/I	Fault	Hit/I	Fault	Fault
<i>IM_ad</i>	Stall/	Stall/	Stall/	Stall/	Stall/	Stall/	Fault	Hit/M	AckNum_set/IM_a	Hit/M	Ack_dec/	Fault
<i>IM_a</i>	Stall/	Stall/	Stall/	SaveReq/IM_aS	SaveReq/IM_aI	Fault	Fault	Fault	Fault	Fault	Ack_dec/	Hit/Ack_dec/M
<i>IM_aS</i>	Stall/	Stall/	Stall/	Fault	Fault	InvAck2Req/IM_aSI	Fault	Fault	Fault	Fault	Ack_dec/	Hit/Ack_dec/Data2Both/S
<i>IM_aSI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Fault	Fault	Fault	Ack_dec/	Hit/Data2Both/Ack_dec/I
<i>IM_aI</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Fault	Fault	Fault	Ack_dec/	Hit/Data2Req/Ack_dec/I
S	Hit/	GetM/SM_ad	PutS/SI_a	Fault	Fault	InvAck2Req/I	Fault	Fault	Fault	Fault	Fault	Fault
<i>SM_ad</i>	Hit/	Stall/	Stall/	Stall/	Stall/	InvAck2Req/IM_ad	Fault	Hit/M	AckNum_set/SM_a	Fault	Ack_dec/	Fault
<i>SM_a</i>	Hit/	Stall/	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Fault	Ack_dec/	Hit/Ack_dec/M
M	Hit/	Hit/	PutM_Data/MI_a	Data2Both/S	Data2Req/I	InvAck_Data/I	Fault	Fault	Fault	Fault	Fault	Fault
<i>MI_a</i>	Stall/	Stall/	Stall/	Data2Both/SI_a	Data2Req/II_a		I	Fault	Fault	Fault	Fault	Fault
<i>SI_a</i>	Stall/	Stall/	Stall/	Fault	Fault	InvAck2Req/II_a	I	Fault	Fault	Fault	Fault	Fault
<i>II_a</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	I	Fault	Fault	Fault	Fault	Fault

B.8 MSI_LLC_directory

Table B.15. MSI_LLC_directory: the numbered lists the table below is written in.

events	actions	states
0 GetS	0 Stall	0 N
1 GetM	1 GetData	1 NS_d
2 Replacement	2 SendData	2 NM_d
3 PutS	3 SaveData	3 I
4 last_PutS	4 SetOwner	4 S
5 InvAck	5 ClearOwner	5 M
6 last_InvAck	6 SendInv	6 S_d
7 PutM_Data_fromOwner	7 WriteBack	7 N_d
8 PutM_Data_fromNonOwner	8 FwdGetS	8 N_a
9 Data_fromLowerInterface	9 FwdGetM	
10 Data_fromUpperInterface	10 PutAck	
	11 IncSharers	
	12 DecSharers	
	13 Fault	

Table B.16. MSI_LLC_directory: transitions as actions/next_state; an empty cell ignores the event, **Fault** aborts the run. **Bold** states are stable, *italic* ones transient.

state	GetS	GetM	Replacement	Puts	last_Puts	InvAck	last_InvAck	PutM_Data_FromOwner	PutM_Data_FromNonOwner	Data_FromLowerInterface	Data_FromUpperInterface
N	GetData/IncSharers/NS_d	SetOwner/GetData/NM_d	Fault	Fault	Fault	Fault	Fault	Fault	PutAck/	Fault	Fault
<i>NS_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Fault	PutAck/	Fault	SendData/S
<i>NM_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	Fault	Fault	PutAck/	Fault	SendData/M
I	SendData/IncSharers/S	SetOwner/SendData/M	WriteBack/N	PutAck/	PutAck/	Fault	PutAck/	Fault	PutAck/	Fault	Fault
S	SendData/IncSharers/	SetOwner/SendData/SendInv/M	SendInv/WriteBack/N_a	PutAck/DecSharers/	PutAck/DecSharers/I	Fault	Fault	Fault	PutAck/DecSharers/	Fault	Fault
M	ClearOwner/FwdGetS/IncSharers/S_d	SetOwner/FwdGetM/	SendInv/N_d	PutAck/	PutAck/	DecSharers/	DecSharers/	ClearOwner/DecSharers/PutAck/SaveData/I	PutAck/DecSharers/	Fault	Fault
<i>S_d</i>	Stall/	Stall/	Stall/	Stall/	Stall/	Fault	Fault	Fault	PutAck/DecSharers/	SaveData/S	Fault
<i>N_d</i>	Stall/	Stall/	Stall/	Fault	Fault	Fault	ClearOwner/DecSharers/WriteBack/N	ClearOwner/DecSharers/PutAck/WriteBack/N	PutAck/DecSharers/	DecSharers/WriteBack/N	Fault
<i>N_a</i>	Stall/	Stall/	Stall/	PutAck/	PutAck/	DecSharers/	DecSharers/N	Fault	PutAck/	Fault	Fault

Chapter C

Command line, scripts and environment

To be written.

Chapter D

File formats

To be written.

Index of configuration keys

addr_filter, 46
arbiter_candidates_ids, 41
arbiter_type, 41–44

buffers_max_size, 42, 43
bus.buffers_max_size, 38
bus.controller_type, 38
bus.interconnect_controller.candidates_id, 38
bus.m_lower_level_ids, 38
bus.upper_level_cache_ids, 38
bus[*].buffers_max_size, 35, 37, 40
bus[0].controller_type, 35, 40
bus[0].interconnect_controller.arbiter_type, 37
bus[0].m_lower_level_ids, 35, 40
bus[0].response_reserve, 35, 40
bus[0].upper_level_cache_ids, 35, 40
bus[1].controller_type, 35, 37, 40
bus[1].interconnect_controller.candidates_id, 35, 37, 40
bus[1].m_lower_level_ids, 35, 37, 40
bus[1].upper_level_cache_ids, 35, 37, 40
bus_type, 35, 37, 39

cache_controller[*].dprint.addr_filter, 36, 38, 39, 41
cache_controller[*].dprint.cond_addr, 36, 38, 39, 41
cache_controller[*].dprint.enable, 36, 38, 41
cache_controller[*].dprint.target, 36, 38, 39, 41
cache_controller[*].fsm_filename, 36, 37, 39, 40
cache_controller[*].m_data_handler.pwb_size, 36, 37, 40
cache_controller[*].m_shared_memory_id, 35, 37, 38, 40
cache_controller[*].num_mshr, 36, 37, 40
cache_controller[*].processing_queue_size, 36, 37, 40
cache_controller[*].protocol_type, 36–38, 40
cache_controller[0].m_id, 36, 37, 39, 40
cache_controller[1].m_id, 36, 37, 39, 40
cache_controller[2].dprint.cond_msg_id, 39
cache_controller[2].dprint.enable, 39
cache_controller[2].m_id, 36, 37, 39, 40
cache_controller[3].m_id, 36, 37, 39, 40
cache_controller_type, 35, 37, 38, 40
cache_size, 45
candidates_id, 44

component_ids, 43
cond_addr, 46
cond_from, 46
cond_msg_id, 46
cond_owner, 46
connection_matrix[0], 43
connection_matrix[1], 43, 44
connection_matrix[2], 43, 44
connection_matrix[3], 43, 44
connection_matrix[4], 43, 44
connection_matrix[5], 44
controller_type, 42, 44

enable, 46

fsm_filename, 41

interconnect.buffers_max_size, 38
interconnect_type, 38

llc_controller.arbiter_candidates_ids, 36, 38, 39, 41
llc_controller.arbiter_type, 36, 38, 39, 41
llc_controller.dprint.addr_filter, 36, 38, 39, 41
llc_controller.dprint.cond_addr, 36, 38, 39, 41
llc_controller.dprint.enable, 36, 38, 39, 41
llc_controller.dprint.target, 36, 38, 39, 41
llc_controller.fsm_filename, 36, 37, 39, 40
llc_controller.m_data_handler.cache_size, 36, 37, 39, 40
llc_controller.m_data_handler.m_data_access_latency, 36, 38–40
llc_controller.m_data_handler.m_ways_count, 36, 37, 39, 40
llc_controller.m_data_handler.pwb_size, 36, 38, 40
llc_controller.m_id, 36, 37, 39, 40
llc_controller.m_shared_memory_id, 36, 37, 39, 40
llc_controller.num_mshr, 36, 38, 40
llc_controller.processing_queue_size, 36, 38, 41
llc_controller.protocol_type, 36, 37, 39, 40
llc_controller_type, 35, 40

m_block_size, 45
m_clk_period, 41–43, 45, 46
m_data_access_latency, 45

m_data_handler.m_data_access_latency, 10
m_data_handler.pwb_size, 10
m_data_latency, 43
m_id, 41, 46
m_llc_id, 46
m_lower_level_ids, 42
m_main_memory.m_id, 36, 38, 39, 41
m_main_memory.m_llc_id, 36, 38, 39, 41
m_memory_latency, 23, 46
m_number_of_OoO_requests, 5, 45
m_request_latency, 42, 43
m_response_latency, 42
m_shared_memory_id, 41
m_ways_count, 45
main_memory_type, 37
mcsim_scheduler, 37

num_cores, 7, 35, 37–39
num_mshr, 10, 42

perfect_llc, 41
print_addr, 46

print_clk, 46
print_from, 46
print_msg_id, 46
print_name, 46
print_owner, 46
print_preamble, 46
print_to, 46
processing_queue_size, 9, 41, 42, 46
protocol_type, 41
pwb_size, 45

replacement_policy_name, 45
response_reserve, 44

switch_ids, 43

target, 46

upper_level_cache_ids, 42

way_partition, 45
workload_path, 35, 37–39

Index

ClearOwner (FSM action), 11

Data2Both (FSM action), 10, 13

GetData (FSM action), 16

GetS (FSM action), 10

IssueInv (FSM action), 10, 11, 13, 16, 18

OCTOPUS_EVENT_DUMP (environment), 25

OCTOPUS_EVENT_DUMP_MIN (environment), 25

SaveData (FSM action), 13

SetOwner (FSM action), 11

WriteBack (FSM action), 18